Decomposition-free variational quantum linear solver: Application in computational mechanics

Article *in* Computer Methods in Applied Mechanics and Engineering · September 2025

DOI: 10.1016/j.cma.2025.118396

2 authors:

Yongchun Xu
Wuhan University
7 PUBLICATIONS **42** CITATIONS
SEE PROFILE

Heng Hu
Wuhan University
105 PUBLICATIONS **2,073** CITATIONS
SEE PROFILE

Accepted by Computer Methods in Applied Mechanics and Engineering, DOI: 10.1016/j.cma.2025.118396

Decomposition-free variational quantum linear solver: Application in computational mechanics

Yongchun Xu[a], Heng Hu[a,b,*]

[a]School of Civil Engineering, Wuhan University, 8 South Road of East Lake, Wuchang, 430072 Wuhan, PR China
[b]School of Mathematics and Statistics, Ningxia University, 750021 Yinchuan, PR China

Abstract

Solving a linear system of equations is a fundamental task in computational mechanics. The recently proposed variational quantum linear solver (VQLS) offers potential acceleration for this task by using quantum computing. However, its application faces a critical bottleneck: the costly requirement to decompose the coefficient matrix into a linear combination of unitary matrices. In this work, we propose a decomposition-free variational quantum linear solver (DF-VQLS) that eliminates this requirement, enabling direct application without matrix decomposition. The key innovation lies in proposing two vectorization techniques, which map the cost functions of VQLS to the inner product of vectors. Specifically, the vectorization techniques reshape the matrix into a vector, and only manipulations on the vector are needed to compute the cost functions, thereby eliminating matrix decomposition entirely. The convergence and accuracy of the proposed method are validated through numerical examples on a quantum simulator. Three application examples in computational mechanics, including bar, truss, and two-dimensional continuum problems, are also presented to show the potential feasibility.

Keywords: Quantum computing; Quantum linear solver; Vectorization; Decomposition-free.

1 Introduction

The solution of linear systems of equations represents a fundamental and ubiquitous challenge in computational mechanics. These systems emerge as the cornerstone of numerical simulations across nearly all disciplines of computational mechanics, including solid mechanics, fluid dynamics, and multiphysics problems. They arise primarily from two sources: (1) the spatial discretization of governing partial differential equations through numerical methods such as the finite element method; and (2) the iterative linearization of nonlinear problems, where complex nonlinear phenomena are approximated through sequences of linear systems. Classical direct linear solvers, such as Gaussian elimination [1], have $\mathcal{O}(N^3)$ time complexity for a system with dimension N, making them prohibitively costly for large-scale problems. While modern iterative techniques, such as the conjugate gradient method [2], offer improved scaling, typically $\mathcal{O}(\sqrt{\kappa}N)$ for sparse systems with condition number κ, their performance remains constrained by matrix properties and problem size. These limitations underscore the need for improved linear solvers in computational mechanics.

Recently, the possibility of applying quantum computing to accelerate solving linear systems has drawn extensive attention [3]. The Harrow-Hassidim-Lloyd (HHL) algorithm [4] is perhaps

*Corresponding author. E-mail address: huheng@whu.edu.cn.

the most well-known quantum algorithm for linear systems, which theoretically enables the preparation of a solution quantum state with $\mathcal{O}(\log N)$ complexity. However, it requires a large number of qubits and a deep circuit depth, making it impractical for current quantum hardware [5]. The variational quantum linear solver (VQLS) proposed by Bravo-Prieto et al. [6] stands out as a promising alternative, which is relatively less dependent on qubit count and circuit depth. Its basic idea is to use a parameterized quantum circuit, referred to as the ansatz, to generate a quantum state proportional to the solution of a linear system. To achieve this, a cost function is defined to quantify the inaccuracy of the solution generated by the ansatz. During the solving process, the cost function is minimized with respect to the parameters in the ansatz, yielding the optimal solution. The advantage of VQLS is its heuristic cost scaling $\mathcal{O}(\text{polylog}(N))$ [6], which is also a significant improvement compared to classical linear solvers. This potential has driven its application to diverse problems, e.g., Ali and Kabel [7] used VQLS in the finite element analysis of the Poisson equation, Trahan et al. [8] applied VQLS to solve time-dependent heat and wave equations, and Balducci et al. [9] applied VQLS to solve tridiagonal linear systems in finite element analysis, among others.

Despite its theoretical promise, VQLS faces a critical limitation in applications: the costly requirement for decomposing the coefficient matrix into a linear combination of unitaries (LCU) [9–11]. This arises because the cost function involves matrix-vector multiplications, which on a quantum computer must be expressed as unitary operations [12]. However, the matrices in typical computational problems, e.g., stiffness matrices in the finite element method, are inherently non-unitary and cannot be easily decomposed into LCU. While prior work has explored decomposition for matrices with special structures [9–11, 13, 14], e.g., sparse, symmetric, periodic, and tridiagonal, these methods may not be generalizable to arbitrary matrices. In contrast, for a generic decomposition method that does not use the special structure, such as the methods in [8, 15], the decomposition process needs to solve an additional linear system of equations, which is costly and may eliminate the advantage of VQLS. In addition, most decomposition methods mentioned above produce at least $\mathcal{O}(N)$ unitary terms for a sparse matrix, requiring $\mathcal{O}(N^2)$ circuit executions per cost function evaluation [6], which may also eliminate the benefits of VQLS.

In this work, we propose a decomposition-free variational quantum linear solver, referred to as DF-VQLS, that bypasses the need for matrix decomposition entirely. The key innovation is the proposal of two vectorization techniques, which reformulate the cost functions of VQLS as the inner product of vectors. In this manner, the coefficient matrix is reshaped as a vector, instead of an LCU from matrix decomposition in the original VQLS. The required inner product computation is achieved via a standard swap test quantum algorithm [16], which does not need the unitary operations corresponding to the LCU, thereby effectively eliminating the need for matrix decomposition. The proposed DF-VQLS holds the following two clear advantages:

- No decomposition is required for the coefficient matrix in the linear system, which eliminates the cost overhead and addresses a critical limitation of the original VQLS.
- Only two circuits are required per cost function evaluation, notably reducing the number of circuit executions of the original VQLS arising from decomposition.

The convergence and accuracy of the proposed method are validated on the quantum simulator Qiskit developed by IBM [17]. Three application examples in computational mechanics, including bar, truss, and two-dimensional continuum problems, are also presented to show the potential feasibility.

The rest of the paper is structured as follows. Section 2 first reviews the original VQLS and

its decomposition bottleneck, then introduces the two vectorization techniques and the swap test algorithm, and finally presents the DF-VQLS circuits that eliminate decomposition. Section 3 validates the convergence and accuracy of the proposed DF-VQLS via solving several linear systems. Section 4 presents application examples in three typical computational mechanics problems. Section 5 provides some conclusions and future directions.

2 Methodology

In this section, we present detailed formulations of the proposed DF-VQLS. We begin by reviewing the original VQLS in Section 2.1, explaining its core components and procedures, especially the computation of the cost functions. As key methods to be used, two vectorization techniques are then proposed in Section 2.2, along with a proof for the standard swap test quantum algorithm. In Section 2.3, by integrating the vectorization techniques and the swap test, we design quantum circuits for computing the cost functions in a decomposition-free scheme, which yields the proposed DF-VQLS.

2.1 Overview of the original VQLS

We consider solving a linear system of equations in the form:

$$\boldsymbol{K}\boldsymbol{u} = \boldsymbol{f}, \tag{1}$$

where $\boldsymbol{K} \in \mathbb{R}^{N\times N}$ and $\boldsymbol{u}, \boldsymbol{f} \in \mathbb{R}^{N}$. Such systems are ubiquitous in computational mechanics. For instance, in finite element analysis of linear elastic solids, $\boldsymbol{K}$ represents the global stiffness matrix, $\boldsymbol{f}$ the force vector, and $\boldsymbol{u}$ the nodal displacements.

For easier understanding, an overview of VQLS is provided in Figure 1. The VQLS aims to solve Eq. (1) by finding a quantum state $|\boldsymbol{u}(\boldsymbol{\theta})\rangle$ such that:

$$\boldsymbol{K}|\boldsymbol{u}(\boldsymbol{\theta})\rangle \propto |\boldsymbol{f}\rangle, \tag{2}$$

where $|\boldsymbol{f}\rangle = \boldsymbol{f}/\|\boldsymbol{f}\|$ is a normalized vector. The state $|\boldsymbol{u}(\boldsymbol{\theta})\rangle$ is generated via a parameterized quantum circuit, referred to as the ansatz:

$$|\boldsymbol{u}(\boldsymbol{\theta})\rangle = \boldsymbol{V}(\boldsymbol{\theta})|0\rangle^{\otimes n_q}, \tag{3}$$

where $\boldsymbol{V}(\boldsymbol{\theta})$ is the ansatz (parameterized by $\boldsymbol{\theta}$), and $n_q = \log_2 N$ qubits encode the state. The mathematical essence of the ansatz is a unitary operator [12], which rotates the ground state $|0\rangle^{\otimes n_q}$ to $|\boldsymbol{u}(\boldsymbol{\theta})\rangle$, controlled by $\boldsymbol{\theta}$. In this work, we adopt the hardware-efficient ansatz from [18], illustrated in Figure 2.

The accuracy of $|\boldsymbol{u}(\boldsymbol{\theta})\rangle$ is evaluated using cost functions based on the cosine similarity between $\boldsymbol{K}|\boldsymbol{u}(\boldsymbol{\theta})\rangle$ and $|\boldsymbol{f}\rangle$. Two types of cost functions are introduced as multiple options [6]. First, the global cost function C_G is defined as:

$$C_G(\boldsymbol{\theta}) = 1 - \frac{|\langle \boldsymbol{f}|\boldsymbol{K}|\boldsymbol{u}(\boldsymbol{\theta})\rangle|^2}{\langle \boldsymbol{u}(\boldsymbol{\theta})|\boldsymbol{K}^T\boldsymbol{K}|\boldsymbol{u}(\boldsymbol{\theta})\rangle}. \tag{4}$$

The numerator measures the inner product (cosine similarity), while the denominator normalizes by the squared norm of $\boldsymbol{K}|\boldsymbol{u}(\boldsymbol{\theta})\rangle$. When Eq. (2) is satisfied, we have $C_G(\boldsymbol{\theta}) \to 0$ [6]. Second,

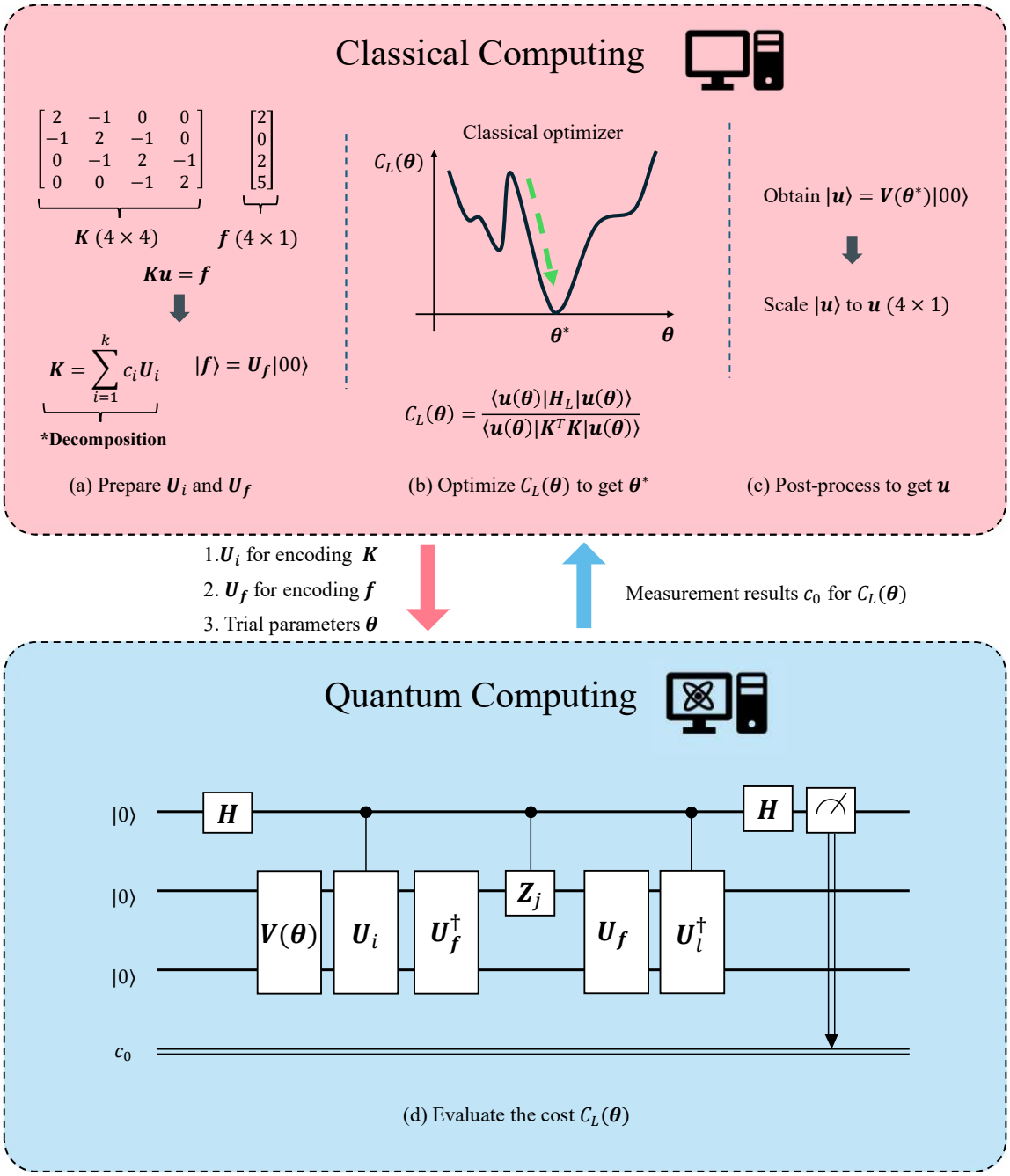


Figure 1: Overview of the VQLS proposed by Bravo-Prieto et al. [6], illustrated for a problem size $N = 4$. Note that the local cost function C_L is used for an example. This hybrid classical-quantum algorithm solves $\boldsymbol{Ku} = \boldsymbol{f}$, where quantum computing evaluates the cost function and classical computing handles matrix decomposition, cost function optimization, and post-processing.

the local cost function C_L, more scalable for large problems, is:

$$C_L(\boldsymbol{\theta}) = \frac{\langle \boldsymbol{u}(\boldsymbol{\theta}) | \boldsymbol{H}_L | \boldsymbol{u}(\boldsymbol{\theta}) \rangle}{\langle \boldsymbol{u}(\boldsymbol{\theta}) | \boldsymbol{K}^T \boldsymbol{K} | \boldsymbol{u}(\boldsymbol{\theta}) \rangle}, \tag{5}$$

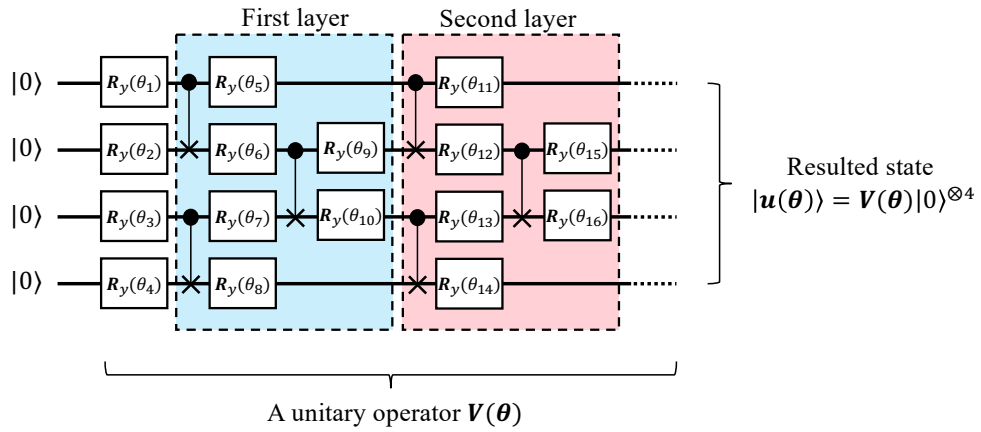


Figure 2: Hardware-efficient ansatz from Kandala et al. [18], shown for $n_q = 4$ qubits (yielding a quantum state $|\boldsymbol{u}(\boldsymbol{\theta})\rangle$ of size $N = 16$). The ansatz initializes with $\boldsymbol{R}_y$ gates (parameterized by θ), followed by entangling layers of CNOT and $\boldsymbol{R}_y$ gates. Increasing the number of layers enhances expressibility.

where the effective Hamiltonian $\boldsymbol{H}_L$ is:

$$\boldsymbol{H}_L = \boldsymbol{K}^T \boldsymbol{U}_{\boldsymbol{f}} \left(\mathbb{I} - \frac{1}{n} \sum_{j=1}^{n} |0_j\rangle\langle 0_j| \otimes \mathbb{I}_{\bar{j}} \right) \boldsymbol{U}_{\boldsymbol{f}}^{\dagger} \boldsymbol{K}, \tag{6}$$

Here, $\boldsymbol{U}_{\boldsymbol{f}}$ is a unitary operator, such that $|\boldsymbol{f}\rangle = \boldsymbol{U}_{\boldsymbol{f}}|0\rangle^{\otimes n_q}$. The dagger notation $\dagger$ refers to the conjugate transpose of a matrix containing complex numbers, the term $\mathbb{I}$ denotes the identity operator, and $|0_j\rangle\langle 0_j| \otimes \mathbb{I}_{\bar{j}}$ is a projection operator acting on all qubits except j. Similar to the global cost function, when Eq. (2) is satisfied, we have $C_L(\boldsymbol{\theta}) \to 0$ [6].

The computations of the C_G and C_L are essentially matrix-vector multiplications. Since quantum computing only supports unitary operations [12], VQLS requires $\boldsymbol{K}$ to be decomposed into a linear combination of unitaries (LCU) to compute C_G or C_L:

$$\boldsymbol{K} = \sum_{i=1}^{k} c_i \boldsymbol{U}_i \tag{7}$$

where $\boldsymbol{U}_i$ are unitary matrices, c_i are coefficients, and k the total number of decomposed unitaries. Substitute Eq. (7) into Eqs. (4) and (5), the cost functions then become:

$$C_G(\boldsymbol{\theta}) = 1 - \frac{\sum_{i=1}^{k}\sum_{l=1}^{k} c_i c_l \langle \boldsymbol{u}(\boldsymbol{\theta})|\boldsymbol{U}_i^{\dagger}|\boldsymbol{f}\rangle\langle \boldsymbol{f}|\boldsymbol{U}_l|\boldsymbol{u}(\boldsymbol{\theta})\rangle}{\sum_{i=1}^{k}\sum_{l=1}^{k} c_i c_l \langle \boldsymbol{u}(\boldsymbol{\theta})|\boldsymbol{U}_i^{\dagger}\boldsymbol{U}_l|\boldsymbol{u}(\boldsymbol{\theta})\rangle} \tag{8a}$$

$$C_L(\boldsymbol{\theta}) = \frac{\sum_{i=1}^{k}\sum_{l=1}^{k} c_i c_l \langle \boldsymbol{u}(\boldsymbol{\theta})|\boldsymbol{U}_i^{\dagger}\boldsymbol{U}_{\boldsymbol{f}} \left(\mathbb{I} - \frac{1}{n}\sum_{j=1}^{n} |0_j\rangle\langle 0_j| \otimes \mathbb{I}_{\bar{j}} \right) \boldsymbol{U}_{\boldsymbol{f}}^{\dagger}\boldsymbol{U}_l |\boldsymbol{u}(\boldsymbol{\theta})\rangle}{\sum_{i=1}^{k}\sum_{l=1}^{k} c_i c_l \langle \boldsymbol{u}(\boldsymbol{\theta})|\boldsymbol{U}_i^{\dagger}\boldsymbol{U}_l|\boldsymbol{u}(\boldsymbol{\theta})\rangle} \tag{8b}$$

To obtain each term in the above expressions (e.g., $\langle \boldsymbol{u}(\boldsymbol{\theta})|\boldsymbol{U}_i^{\dagger}\boldsymbol{U}_l|\boldsymbol{u}(\boldsymbol{\theta})\rangle$), Bravo-Prieto et al. designed several quantum circuits primarily based on the Hadamard test (see details in [6]). For instance, the quantum circuit for computing the terms in $C_L(\boldsymbol{\theta})$ is shown in Figure 1(d), where the total number of qubits required is $n_q + 1$. Note that the decomposed unitaries, i.e., $\boldsymbol{U}_i$ and $\boldsymbol{U}_l$ in the circuit, are implemented as quantum gates to achieve matrix-vector multiplications. From Eq. (8), it is evident that to evaluate $C_G(\boldsymbol{\theta})$ or $C_L(\boldsymbol{\theta})$ once, the number of circuits that

need to be executed grows quadratically with the number of decomposed unitary terms k, i.e., the circuit execution complexity is $\mathcal{O}(k^2)$ per cost evaluation. This complexity arises from the double summation over the k unitaries $\boldsymbol{U}_i$.

Based on the computation of the cost functions described above, the VQLS solving process uses a classical optimizer to minimize either $C_G(\boldsymbol{\theta})$ or $C_L(\boldsymbol{\theta})$. As shown in Figure 1(b) (where $C_L(\boldsymbol{\theta})$ serves as an example), the quantum computer obtains the cost function and returns the result to the classical optimizer, which then iteratively updates $\boldsymbol{\theta}$ to minimize $C_L(\boldsymbol{\theta})$. This minimization process is achieved using a classical optimizer like gradient descent [6, 19] or COBYLA [20]. The optimization iterative process continues until meeting convergence criteria, yielding an optimized $|\boldsymbol{u}(\boldsymbol{\theta}^*)\rangle$ that approximates a normalized solution to Eq. (1). In the post-processing stage (shown in Figure 1(c)), the optimal $|\boldsymbol{u}(\boldsymbol{\theta}^*)\rangle$ is read out from the ansatz. To recover the original scale of the true solution $\boldsymbol{u}$ from the unit quantum state vector $|\boldsymbol{u}(\boldsymbol{\theta}^*)\rangle$, one can perform least-squares fitting to determine a scale factor s that minimizes the l_2-norm $||\boldsymbol{f} - s\boldsymbol{K}\,|\boldsymbol{u}(\boldsymbol{\theta}^*)\rangle\,||$. The final quantum solution is then obtained as $\boldsymbol{u} = s\,|\boldsymbol{u}(\boldsymbol{\theta}^*)\rangle$.

Though VQLS has a promising heuristic cost $\mathcal{O}(\text{polylog}(N))$ [6], it encounters limitations arising from the decomposition $\boldsymbol{K} = \sum_{i=1}^{k} c_i \boldsymbol{U}_i$. First, general matrices without a special structure cannot be easily decomposed into an LCU, and this introduces additional costs that eliminate the quantum advantage. Second, many existing decomposition methods produce at least $k \sim \mathcal{O}(N)$ unitary terms [9, 10, 13], meaning the circuit execution complexity becomes at least $\mathcal{O}(N^2)$ per cost evaluation, which also potentially eliminates the quantum advantage. In the following sections, we will overcome these limitations by proposing a decomposition-free variational quantum linear solver (DF-VQLS), where the matrix decomposition is no longer required.

2.2 Vectorization and swap test

In this subsection, we propose two vectorization techniques, serving as the foundation for developing DF-VQLS. Specifically, we demonstrate that the matrix-vector multiplications in C_G and C_L can be mapped to the inner product of vectors, thereby eliminating the need for explicit matrix operations. Additionally, we also briefly introduce the swap test quantum algorithm proposed by Buhrman et al. [16], which is used to achieve such an inner product on a quantum computer.

The core definition to be used is the vectorization of a matrix [21]. For any $\boldsymbol{A} \in \mathbb{R}^{M\times N}$, the vectorization operator $\text{vec}(\boldsymbol{A})$ produces a column vector in $\mathbb{R}^{MN}$ by stacking matrix columns sequentially. For example:

$$\boldsymbol{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \Rightarrow \text{vec}(\boldsymbol{A}) = \begin{bmatrix} a \\ c \\ b \\ d \end{bmatrix} \tag{9}$$

A more formal definition is:

$$\text{vec}(\boldsymbol{A}) = \sum_{i=1}^{M}\sum_{j=1}^{N} A_{ij}(\boldsymbol{e}_j \otimes \boldsymbol{e}_i) \tag{10}$$

Where $\{\boldsymbol{e}_p\}_{p=1}^{N}$ be the standard basis vectors in $\mathbb{R}^N$. With this definition, we present the two vectorization techniques below.

First, we classify two types of matrix-vector multiplications used by C_G and C_L. As shown in Eqs. (4) and (5), the numerators in both cost functions involve a bilinear form computation: $\boldsymbol{b}^T\boldsymbol{A}\boldsymbol{x}$, where $\boldsymbol{A} \in \mathbb{R}^{M\times N}$, $\boldsymbol{b} \in \mathbb{R}^M$, and $\boldsymbol{x} \in \mathbb{R}^N$. The denominators involve a quadratic form computation: $\boldsymbol{x}^T\boldsymbol{M}\boldsymbol{R}\boldsymbol{x}$, where $\boldsymbol{M}, \boldsymbol{R} \in \mathbb{R}^{N\times N}$. Next, we show that these two computations can be mapped to vector inner products by two vectorization techniques. Note that the vectorization techniques are independent of VQLS cost functions. Therefore, we adopt different notations of the matrices and vectors to emphasize generality.

Theorem 1. *For any matrix* $\boldsymbol{A} \in \mathbb{R}^{M\times N}$*, vectors* $\boldsymbol{b} \in \mathbb{R}^M$ *and* $\boldsymbol{x} \in \mathbb{R}^N$*:*

$$\boldsymbol{b}^T\boldsymbol{A}\boldsymbol{x} = vec(\boldsymbol{A})^T(\boldsymbol{x} \otimes \boldsymbol{b}) \tag{11}$$

Proof. We begin by expressing the vectorization and Kronecker product in terms of standard basis vectors:

$$\text{vec}(\boldsymbol{A}) = \sum_{i=1}^{M}\sum_{j=1}^{N} A_{ij}(\boldsymbol{e}_j \otimes \boldsymbol{e}_i) \tag{12}$$

and:

$$\boldsymbol{x} \otimes \boldsymbol{b} = \left(\sum_{j=1}^{N} x_j\boldsymbol{e}_j\right) \otimes \left(\sum_{i=1}^{M} b_i\boldsymbol{e}_i\right) = \sum_{j=1}^{N}\sum_{i=1}^{M} x_j b_i(\boldsymbol{e}_j \otimes \boldsymbol{e}_i) \tag{13}$$

Computing the inner product through basis vector contractions:

$$\text{vec}(\boldsymbol{A})^T(\boldsymbol{x} \otimes \boldsymbol{b}) = \left(\sum_{i,j} A_{ij}(\boldsymbol{e}_j \otimes \boldsymbol{e}_i)^T\right)\left(\sum_{k,l} x_k b_l(\boldsymbol{e}_k \otimes \boldsymbol{e}_l)\right) \tag{14}$$

Expanding using the linearity of the inner product:

$$= \sum_{i,j}\sum_{k,l} A_{ij}x_k b_l(\boldsymbol{e}_j^T\boldsymbol{e}_k)(\boldsymbol{e}_i^T\boldsymbol{e}_l) \tag{15}$$

Applying orthonormality of basis vectors ($\boldsymbol{e}_a^T\boldsymbol{e}_b = \delta_{ab}$):

$$= \sum_{i,j}\sum_{k,l} A_{ij}x_k b_l\delta_{jk}\delta_{il} \tag{16}$$

Collapsing sums via Kronecker delta constraints:

$$= \sum_{i,j} A_{ij}x_j b_i \tag{17}$$

Recognizing the final expression as matrix-vector multiplication:

$$= \sum_{i=1}^{M}\sum_{j=1}^{N} b_i A_{ij}x_j = \boldsymbol{b}^T\boldsymbol{A}\boldsymbol{x} \tag{18}$$

where the theorem is proved. □

It is worth noting that Roth proved a related theorem [22]. For three matrices $\boldsymbol{E}$, $\boldsymbol{G}$, and

$\boldsymbol{L}$, the equality $\text{vec}(\boldsymbol{EGL}) = (\boldsymbol{L}^T \otimes \boldsymbol{E})\text{vec}(\boldsymbol{G})$ holds. This theorem overlaps with Theorem 1. Specifically, Theorem 1 can be regarded as a special case, where instead of producing a vector, it focuses on a scalar by setting $\boldsymbol{E}$ and $\boldsymbol{L}$ as two vectors.

Theorem 2. *For any matrices $\boldsymbol{M}, \boldsymbol{R} \in \mathbb{R}^{N\times N}$ and vector $\boldsymbol{x} \in \mathbb{R}^N$:*

$$\boldsymbol{x}^T \boldsymbol{M}\boldsymbol{R}\boldsymbol{x} = \big(\boldsymbol{x} \otimes \text{vec}(\boldsymbol{M})\big)^T \big(\text{vec}(\boldsymbol{R}) \otimes \boldsymbol{x}\big) \tag{19}$$

Proof. We first express the vectorizations and Kronecker products in terms of basis vectors:

$$\text{vec}(\boldsymbol{M}) = \sum_{i,j=1}^{N} M_{ij}(\boldsymbol{e}_j \otimes \boldsymbol{e}_i), \quad \text{vec}(\boldsymbol{R}) = \sum_{k,l=1}^{N} R_{kl}(\boldsymbol{e}_l \otimes \boldsymbol{e}_k) \tag{20}$$

Expand the left factor using Kronecker product linearity:

$$\boldsymbol{x} \otimes \text{vec}(\boldsymbol{M}) = \left(\sum_{p=1}^{N} x_p \boldsymbol{e}_p\right) \otimes \left(\sum_{i,j=1}^{N} M_{ij}(\boldsymbol{e}_j \otimes \boldsymbol{e}_i)\right) = \sum_{p,i,j} x_p M_{ij}(\boldsymbol{e}_p \otimes \boldsymbol{e}_j \otimes \boldsymbol{e}_i) \tag{21}$$

Similarly expand the right factor:

$$\text{vec}(\boldsymbol{R}) \otimes \boldsymbol{x} = \left(\sum_{k,l=1}^{N} R_{kl}(\boldsymbol{e}_l \otimes \boldsymbol{e}_k)\right) \otimes \left(\sum_{m=1}^{N} x_m \boldsymbol{e}_m\right) = \sum_{k,l,m} R_{kl} x_m (\boldsymbol{e}_l \otimes \boldsymbol{e}_k \otimes \boldsymbol{e}_m) \tag{22}$$

Compute the inner product through basis vector contractions:

$$\big(\boldsymbol{x} \otimes \text{vec}(\boldsymbol{M})\big)^T \big(\text{vec}(\boldsymbol{R}) \otimes \boldsymbol{x}\big) = \sum_{\substack{p,i,j \\ k,l,m}} x_p M_{ij} R_{kl} x_m \underbrace{\langle \boldsymbol{e}_p \otimes \boldsymbol{e}_j \otimes \boldsymbol{e}_i, \boldsymbol{e}_l \otimes \boldsymbol{e}_k \otimes \boldsymbol{e}_m \rangle}_{\delta_{pl}\delta_{jk}\delta_{im}} \tag{23}$$

Apply orthonormality of basis vectors ($\boldsymbol{e}_a^T \boldsymbol{e}_b = \delta_{ab}$):

$$= \sum_{p,i,j} x_p M_{ij} R_{jp} x_i \quad \text{(after collapsing sums via Kronecker deltas)} \tag{24}$$

Recognize the matrix product structure:

$$= \sum_{i,p=1}^{N} x_p \left(\sum_{j=1}^{N} M_{ij} R_{jp}\right) x_i = \sum_{i,p=1}^{N} x_p (\boldsymbol{M}\boldsymbol{R})_{ip} x_i = \boldsymbol{x}^T \boldsymbol{M}\boldsymbol{R}\boldsymbol{x} \tag{25}$$

This completes the proof. □

By applying the two vectorization techniques in Theorems 1 and 2, both matrix-vector multiplications in the cost functions can be mapped to the inner product of two vectors. This mapping only requires reshaping the matrix into a vector, and it eliminates the need for explicit matrix operations.

Finally, we present a proof of the swap test, which is a standard quantum algorithm for computing the inner product between two quantum states [16]. It is worth noting that the swap test has been applied to distance estimation in data-driven computational mechanics [23]. In

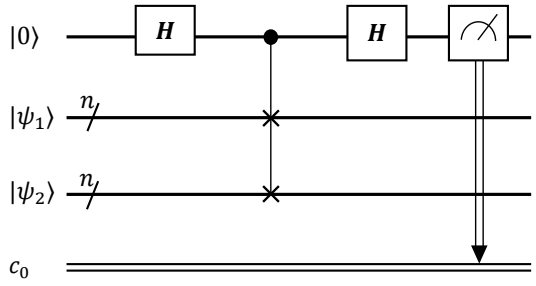


Figure 3: Quantum circuit of the swap test [16], used for computing the inner product between two quantum states $|\psi_1\rangle$ and $|\psi_2\rangle$.

this work, we will show that, integrated with the vectorization techniques above, the swap test can also be naturally applied to compute the cost functions in a decomposition-free scheme. This will be detailed in the next section.

Theorem 3. *Let $|\psi_1\rangle$ and $|\psi_2\rangle$ be two n-qubit quantum states prepared on $2n$ qubits. The squared inner product $|\langle\psi_1|\psi_2\rangle|^2$ can be obtained using the swap test with time complexity $\mathcal{O}(n)$.*

Proof. For easier understanding, the quantum circuit for the swap test is shown in Figure 3. Let the quantum circuit operate on three registers: an ancilla qubit initialized to $|0\rangle$, and two n-qubit registers storing $|\psi_1\rangle$ and $|\psi_2\rangle$. The circuit proceeds as follows:

$$|\Phi_0\rangle = |0\rangle \otimes |\psi_1\rangle \otimes |\psi_2\rangle \tag{26}$$

Apply Hadamard gate to the ancilla:

$$|\Phi_1\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |\psi_1\rangle \otimes |\psi_2\rangle \tag{27}$$

Apply a controlled-SWAP (Fredkin) gate between the two n-qubit registers:

$$|\Phi_2\rangle = \frac{1}{\sqrt{2}}|0\rangle \otimes |\psi_1\rangle|\psi_2\rangle + \frac{1}{\sqrt{2}}|1\rangle \otimes |\psi_2\rangle|\psi_1\rangle \tag{28}$$

Apply the second Hadamard gate to the ancilla:

$$|\Phi_3\rangle = \frac{1}{2}|0\rangle \otimes (|\psi_1\rangle|\psi_2\rangle + |\psi_2\rangle|\psi_1\rangle) + \frac{1}{2}|1\rangle \otimes (|\psi_1\rangle|\psi_2\rangle - |\psi_2\rangle|\psi_1\rangle) \tag{29}$$

Measure the ancilla qubit. The probability of observing $|0\rangle$ is:

$$P(0) = \left\| \frac{1}{2}(|\psi_1\rangle|\psi_2\rangle + |\psi_2\rangle|\psi_1\rangle) \right\|^2 = \frac{1}{2}\left(1 + |\langle\psi_1|\psi_2\rangle|^2\right) \tag{30}$$

Rearranging gives the desired quantity:

$$|\langle\psi_1|\psi_2\rangle|^2 = 2P(0) - 1 \tag{31}$$

The time complexity is dominated by the n parallel controlled-SWAP gates between the n-qubit registers, each implementable with $\mathcal{O}(1)$ elementary gates. Including the constant-time Hadamard operations and measurement, the total gate complexity is $\mathcal{O}(n)$. □

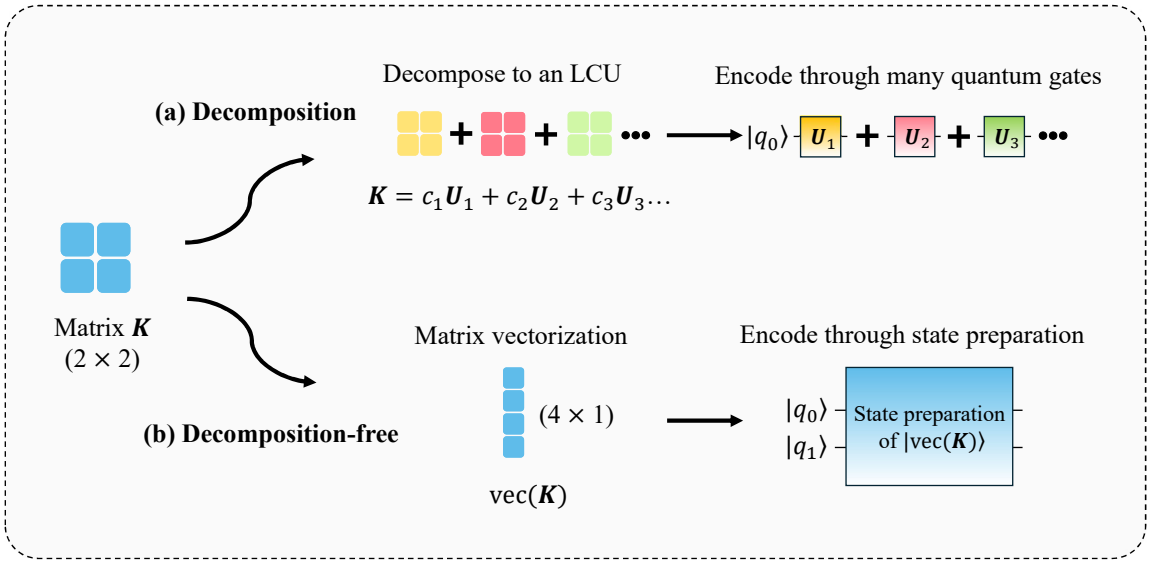


Figure 4: High-level diagram of the decomposition-free scheme, with the matrix decomposition used in the original VQLS as a comparison. (a) Matrix decomposition. (b) Decomposition-free via vectorization.

In summary, we have proposed two vectorization techniques that transform two types of matrix-vector multiplications into vector inner products. We have additionally provided the proof of the swap test quantum algorithm for inner product computation. Building on these foundations, the following section will present the proposed DF-VQLS.

2.3 Decomposition-free quantum circuits for the cost functions

In this section, we introduce the proposed DF-VQLS, which computes the cost functions in a decomposition-free scheme.

A high-level diagram of the decomposition-free scheme is presented in Figure 4, with the matrix decomposition used in the original VQLS as a comparison. In the original VQLS, the coefficient matrix $\boldsymbol{K}$ is decomposed into an LCU. The cost functions are then computed via quantum gates corresponding to these unitaries, as previously detailed in Eq. (8) and Figure 1(d). In contrast, the proposed DF-VQLS no longer needs to decompose the matrix $\boldsymbol{K}$. It only reshapes the matrix into a vector through vectorization and encodes the vector into a quantum state. The cost functions are then computed using the swap test, as derived below.

First, we introduce the quantum circuit for computing the numerator of C_G, i.e., the term $|\langle \boldsymbol{f}|\boldsymbol{K}|\boldsymbol{u}(\boldsymbol{\theta})\rangle|^2$ in Eq. (4). Applying the vectorization technique in Theorem 1, we have:

$$\langle \boldsymbol{f}|\boldsymbol{K}|\boldsymbol{u}(\boldsymbol{\theta})\rangle = \text{vec}(\boldsymbol{K})^T|\boldsymbol{u}(\boldsymbol{\theta}), \boldsymbol{f}\rangle \tag{32}$$

where $|\boldsymbol{u}(\boldsymbol{\theta}), \boldsymbol{f}\rangle = |\boldsymbol{u}(\boldsymbol{\theta})\rangle \otimes |\boldsymbol{f}\rangle$ is a product state [12]. In this manner, the matrix-vector multiplications are transformed into the inner product of two vectors.

Now, we consider achieving such inner product computation using the swap test in Theorem 3. Recall that a swap test computes the squared inner product $|\langle\psi_1|\psi_2\rangle|^2$ between two quantum states $|\psi_1\rangle$ and $|\psi_2\rangle$. We then reformulate the vectorized matrix $\text{vec}(\boldsymbol{K})$ as a quantum

state by normalizing it:

$$|\text{vec}(\boldsymbol{K})\rangle = \frac{1}{\|\boldsymbol{K}\|}\text{vec}(\boldsymbol{K}) = \frac{1}{\|\boldsymbol{K}\|}\sum_{i=0}^{N-1}\sum_{j=0}^{N-1} K_{i+1,j+1}|ji\rangle \tag{33}$$

where $\|\boldsymbol{K}\|$ is the Frobenius norm and $|ji\rangle = \boldsymbol{e}_{j+1}\otimes\boldsymbol{e}_{i+1} = |j\rangle\otimes|i\rangle$. The use of 0-based indexing (e.g., $|ji\rangle$ with $j, i = 0, 1, ...$) instead of 1-based indexing (e.g., $|ji\rangle$ with $j, i = 1, 2, ...$) aligns with the convention of basis states in quantum computing. Note that this normalization is conducted on a classical computer, and it is typically low-cost for modest N. In addition, the dimension of $|\text{vec}(\boldsymbol{K})\rangle$ is $N^2 \times 1$, but it only needs $2n_q$ qubits to encode using quantum superposition, which reflects the unique advantage of quantum computing. Substituting Eq. (33) into Eq. (32), we have:

$$|\langle \boldsymbol{f}|\boldsymbol{K}|\boldsymbol{u}(\boldsymbol{\theta})\rangle|^2 = \|\boldsymbol{K}\|^2\,|\langle \text{vec}(\boldsymbol{K})|\boldsymbol{u}(\boldsymbol{\theta}), \boldsymbol{f}\rangle|^2 \tag{34}$$

It is evident that the term $|\langle \text{vec}(\boldsymbol{K})|\boldsymbol{u}(\boldsymbol{\theta}), \boldsymbol{f}\rangle|^2$ can be naturally computed using the swap test. In the original swap test quantum circuit shown in Figure 3, we replace $|\psi_1\rangle$ and $|\psi_2\rangle$ with $|\text{vec}(\boldsymbol{K})\rangle$ and $|\boldsymbol{u}(\boldsymbol{\theta}), \boldsymbol{f}\rangle$, respectively, resulting in the quantum circuit shown in Figure 5(a). The number of qubits required is $4n_q + 1$, which is larger than the original VQLS but still scales as $\mathcal{O}(\log N)$. Importantly, by executing this circuit, we can obtain the numerator of C_G without the need to decompose the matrix $\boldsymbol{K}$, resulting in the decomposition-free scheme. Meanwhile, another advantage is that only a single quantum circuit is required for computing the numerator. In contrast, the original VQLS requires a circuit execution complexity $\mathcal{O}(k^2)$ per cost evaluation due to the k matrix decomposition unitary terms [6]. These two advantages also hold for the following quantum circuits.

Second, we introduce the quantum circuit for computing the numerator of C_L, i.e., the term $\langle \boldsymbol{u}(\boldsymbol{\theta})|\boldsymbol{H}_L|\boldsymbol{u}(\boldsymbol{\theta})\rangle$ in Eq. (5). The matrix $\boldsymbol{H}_L$ is prepared on a classical computer according to Eq. (6) with a relatively low cost: since typically $|\boldsymbol{f}\rangle \in \mathbb{R}^N$, the unitary operator $\boldsymbol{U}_{\boldsymbol{f}}$ for preparing $\boldsymbol{H}_L$ can be cheaply obtained with $\boldsymbol{U}_{\boldsymbol{f}} = \boldsymbol{I} - \beta\boldsymbol{v}\boldsymbol{v}^T$, where $\boldsymbol{v} = |0\rangle^{\otimes n_q} - |\boldsymbol{f}\rangle$ and $\beta = 2/(\boldsymbol{v}^T\boldsymbol{v})$ [24]. Next, by applying the vectorization technique in Theorem 1, the procedures are almost the same as those in Eqs. (32) to (34). For the sake of simplicity, we omit the derivation and directly provide the final expression:

$$\langle \boldsymbol{u}(\boldsymbol{\theta})|\boldsymbol{H}_L|\boldsymbol{u}(\boldsymbol{\theta})\rangle = \|\boldsymbol{H}_L\|\sqrt{|\langle \text{vec}(\boldsymbol{H}_L)|\boldsymbol{u}(\boldsymbol{\theta}), \boldsymbol{u}(\boldsymbol{\theta})\rangle|^2} \tag{35}$$

The term $|\langle \text{vec}(\boldsymbol{H}_L)|\boldsymbol{u}(\boldsymbol{\theta}), \boldsymbol{u}(\boldsymbol{\theta})\rangle|^2$ is also obtained using the swap test by replacing $|\psi_1\rangle$ and $|\psi_2\rangle$ with $|\text{vec}(\boldsymbol{H}_L)\rangle$ and $|\boldsymbol{u}(\boldsymbol{\theta}), \boldsymbol{u}(\boldsymbol{\theta})\rangle$, respectively, resulting in the quantum circuit shown in Figure 5(b). The number of qubits required is also $4n_q + 1$, which scales as $\mathcal{O}(\log N)$.

Finally, we introduce the quantum circuit for computing the denominator of both C_G and C_L, i.e., the term $\langle \boldsymbol{u}(\boldsymbol{\theta})|\boldsymbol{K}^T\boldsymbol{K}|\boldsymbol{u}(\boldsymbol{\theta})\rangle$ in both Eqs. (4) and (5). By applying the vectorization technique in Theorem 2 and considering Eq. (33):

$$\langle \boldsymbol{u}(\boldsymbol{\theta})|\boldsymbol{K}^T\boldsymbol{K}|\boldsymbol{u}(\boldsymbol{\theta})\rangle = \|\boldsymbol{K}\|^2\sqrt{|\langle \boldsymbol{u}(\boldsymbol{\theta}), \text{vec}(\boldsymbol{K}^T)|\text{vec}(\boldsymbol{K}), \boldsymbol{u}(\boldsymbol{\theta})\rangle|^2} \tag{36}$$

Similarity, the right-hand side term $\left|\langle \boldsymbol{u}(\boldsymbol{\theta}), \text{vec}(\boldsymbol{K}^T)|\text{vec}(\boldsymbol{K}), \boldsymbol{u}(\boldsymbol{\theta})\rangle\right|^2$ can be obtained via the swap test by replacing $|\psi_1\rangle$ and $|\psi_2\rangle$ with $|\boldsymbol{u}(\boldsymbol{\theta}), \text{vec}(\boldsymbol{K}^T)\rangle$ and $|\text{vec}(\boldsymbol{K}), \boldsymbol{u}(\boldsymbol{\theta})\rangle$, respectively, resulting in the quantum circuit shown in Figure 5(c). The number of qubits required is $6n_q + 1$, which scales as $\mathcal{O}(\log N)$. With all of the circuits in Figure 5, we are able to compute both C_G

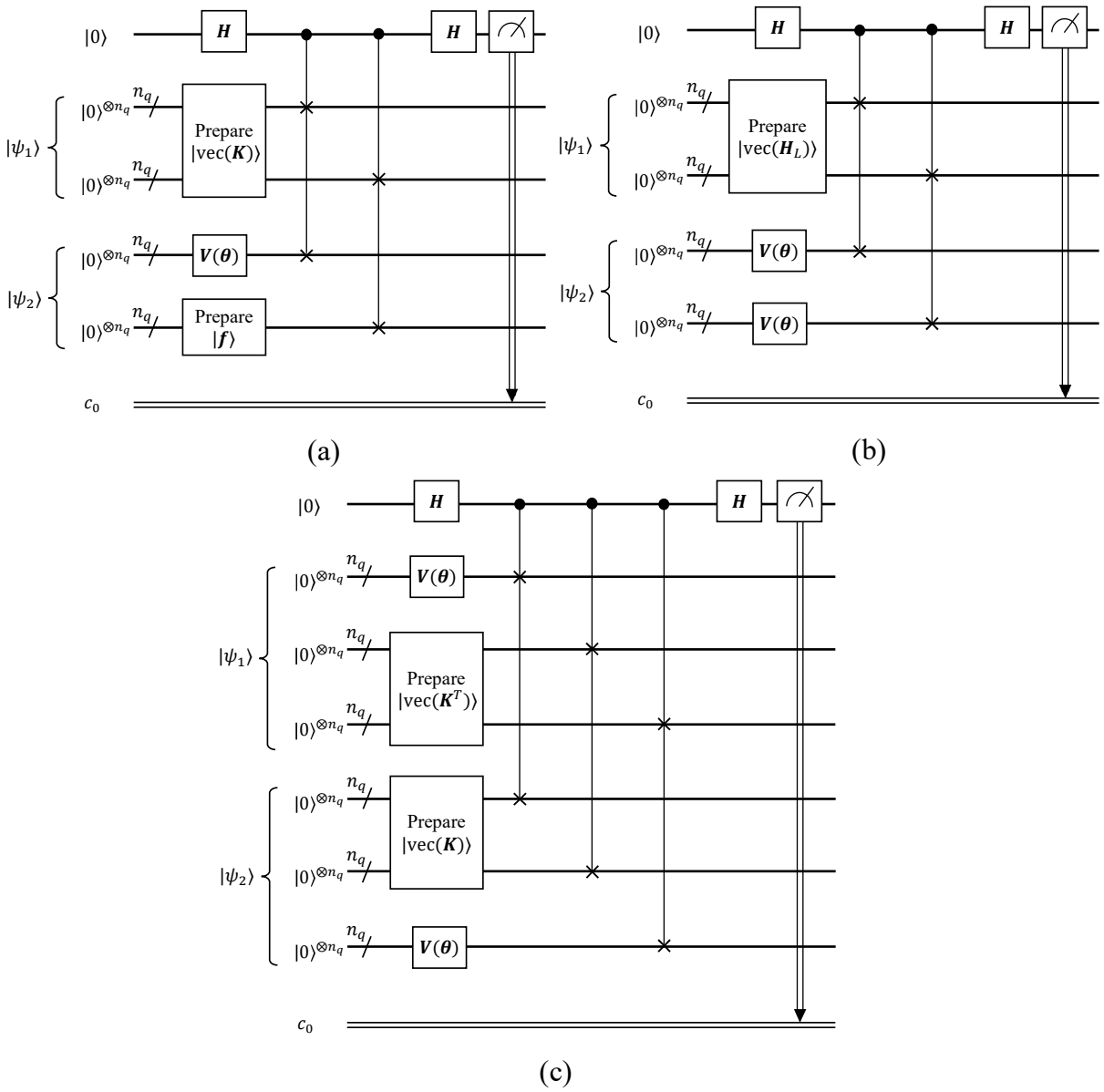


Figure 5: Quantum circuits of the proposed DF-VQLS, which compute the cost functions C_G and C_L in a decomposition-free scheme. (a) Circuit for computing the numerator of C_G. (b) Circuit for computing the numerator of C_L. (c) Circuit for computing the denominators of both C_G and C_L.

and C_L in a decomposition-free scheme, resulting in the DF-VQLS.

In summary, we have proposed DF-VQLS that eliminates the need for matrix decomposition. Compared to the original VQLS, the proposed DF-VQLS provides two clear advantages. First, no matrix decomposition is needed. The matrix simply needs to be reshaped into a vector, followed by a swap test, instead of performing unitary gate operations corresponding to an LCU from decomposition. Therefore, the costly matrix decomposition is eliminated, overcoming a critical limitation of the original VQLS. Second, the number of circuit executions is significantly reduced. In the original VQLS, the number of circuits required per cost evaluation scales quadratically with the number of decomposed unitary terms. In contrast, the proposed DF-VQLS only requires two circuits per cost evaluation, i.e., one for the numerator and another for the denominator. These advantages suggest that the proposed DF-VQLS offers a promising

alternative for quantum linear solver applications.

Remark 1. The vectorization techniques are particularly suitable for quantum computing due to three inherent advantages. First, the vectorized cost function simplifies quantum circuit design. By translating matrix-vector multiplications into inner products, it allows direct computation via the standard swap test, eliminating the need for custom quantum circuits to handle matrix operations. Second, it is interesting to find that the vectorization formulation leverages the natural tensor product structure of quantum states. For instance, the state $|\text{vec}(\boldsymbol{K}), \boldsymbol{u}(\boldsymbol{\theta})\rangle = |\text{vec}(\boldsymbol{K})\rangle \otimes |\boldsymbol{u}(\boldsymbol{\theta})\rangle$ is a product state [12], enabling independent initialization of $|\text{vec}(\boldsymbol{K})\rangle$ and $|\boldsymbol{u}(\boldsymbol{\theta})\rangle$. The combined state is constructed without additional quantum operations, directly aligning with the mathematical framework of quantum computing. Third, both vectorization techniques require only estimating a single scalar value rather than reconstructing a full quantum state. This avoids the quantum readout problem [25, 26] during the optimization process, as only a single probability estimation is required for each quantum circuit.

Remark 2. Efficient state preparation is a necessary requirement for the proposed DF-VQLS, as the right-hand side vector $|\boldsymbol{f}\rangle$ and the vectorized matrix $|\text{vec}(\boldsymbol{K})\rangle$ are encoded to the amplitudes of quantum states. This requirement inherits the well-known quantum input problem [25, 26], a fundamental challenge across quantum computing for scientific computations. Existing quantum linear solvers, including HHL [4] and VQLS, typically assume that a normalized classical vector with N entries can be encoded into a quantum state with $\mathcal{O}(\log N)$ time complexity. While quantum random access memory (qRAM) [27] has been proposed as a theoretical solution for such efficient state preparation, its practical realization remains open to date.

To provide a quantitative estimate of this cost, we analyze the state-of-the-art sparse state preparation algorithm by Zhang et al. [28], which can prepare an n-qubit, d-sparse state with circuit depth $\tilde{\mathcal{O}}(\log(nd))$ and $\mathcal{O}(nd \log d)$ ancillary qubits. For the task of preparing $|\text{vec}(\boldsymbol{K})\rangle$, the number of system qubits is $n = 2n_q = 2\log_2 N$, and the sparsity is the number of non-zero entries in $\boldsymbol{K}$, denoted by S. Let us consider a moderately large linear system with $N = 1024$ ($n_q = 10$), where the stiffness matrix has a sparsity of $S \approx 9N \approx 9216$. Applying the algorithm [28] requires $2n_q = 20$ qubits and achieves a remarkably low, polylogarithmic circuit depth of $\tilde{\mathcal{O}}(\log((2n_q)S)) \approx \tilde{\mathcal{O}}(17.5)$. However, the cost in ancillary qubits is $\mathcal{O}((2n_q)S \log S) \approx \mathcal{O}(2.4 \times 10^6)$, which is overly large for near-term hardware. Therefore, the practical realization of DF-VQLS relies on future breakthroughs in state preparation and quantum hardware.

In this context, we believe the practical feasibility of DF-VQLS on near-term devices may be potentially problem-dependent. For a general sparse matrix, the total qubit cost would be dominated by state preparation, which can be very large as discussed above. However, for specific problems where the data pattern of $\boldsymbol{K}$ and $\boldsymbol{f}$ allows efficient state preparation, a potential feasibility may exist. For instance, a uniform vector can be prepared with a single layer of Hadamard gates. If an application involves such similar patterned data, solving a problem of size $N = 1024$ would require 61 qubits, a number that is acceptable by near-term quantum hardware. We believe this is an interesting and valuable research direction to explore in the future.

Remark 3. While variational quantum algorithms are generally regarded as more suitable for near-term quantum hardware compared to fault-tolerant methods [5], implementing the proposed DF-VQLS on real quantum computers still requires addressing two key challenges related to quantum resources: qubit overhead and circuit depth. These factors currently limit the feasibility of DF-VQLS on noisy intermediate-scale quantum (NISQ) devices [29].

First, the qubit overhead presents a trade-off between the circuit executions. DF-VQLS

increases the qubit count from $n_q + 1$ to $6n_q + 1$, which, on NISQ hardware, leads to a higher error for a single circuit execution due to increased gate count and crosstalk [30]. In return, the number of circuit executions per cost function evaluation is reduced from $\mathcal{O}(k^2)$ to a constant of two. The total computational error is influenced by both the error from a single execution and the number of circuit executions. Therefore, DF-VQLS may become advantageous when the benefit of reducing executions outweighs the cost of increased single-shot error. This condition is potentially met when the number of decomposition terms k is large.

Second, the swap test relies heavily on controlled-SWAP gates, which may significantly increase circuit depth after gate compilation. However, this limitation may potentially be overcome by a more NISQ-friendly method called classical shadow [31]. This method replaces the single deep circuit of the swap test with many repetitions of shallow circuits, each involving only single-qubit randomized Pauli measurements. The inner product is then estimated via efficient classical post-processing. This approach shifts the primary resource cost from circuit depth to the number of measurements, and it presents an interesting future research direction.

Remark 4. It is important to contextualize the potential advantage of DF-VQLS in comparison to classical linear solvers. The heuristic time complexity of DF-VQLS, inheriting from the original VQLS, is $\mathcal{O}(\text{polylog}(N))$. The state-of-the-art classical algorithm for such problems, particularly those arising from the finite element method, is the conjugate gradient method [2]. The complexity of the conjugate gradient method is $\mathcal{O}(N\sqrt{\kappa})$ for a sparse matrix. This suggests that DF-VQLS could offer a potential speedup for problems where the system size N is large, such that $\text{polylog}(N)$ becomes significantly smaller than $N\sqrt{\kappa}$. Moreover, it should be clearly acknowledged that this analysis is based on an ideal evaluation. The realization of the practical advantage of variational quantum computing remains open in the field currently [5,32]. Considering there are many open questions and challenges to be overcome, e.g., limited qubits, hardware error, efficient state preparation, and barren plateaus, it is still not clear to predict a reasonable and specific threshold system size for practical advantage at this stage.

3 Validation

In this section, we validate the proposed DF-VQLS using the quantum simulator Qiskit [17]. Our primary objectives are to: (1) verify its ability to accurately solve linear systems without matrix decomposition, and (2) compare its performance with the original VQLS.

The numerical setup consists of the following components. The Qiskit *statevector* backend executes the quantum circuits. The hardware-efficient ansatz from Figure 2 implements the unitary operator $\boldsymbol{V}(\boldsymbol{\theta})$. The Qiskit built-in method [33] prepares the quantum states for the right-hand side vector $|\boldsymbol{f}\rangle$ and the vectorized matrix $|\text{vec}(\boldsymbol{K})\rangle$. For cost function minimization, the BFGS optimizer from SciPy [19, 34] is adopted, which is a gradient-based optimization algorithm.

The first test case involves solving a linear system with a characteristic matrix $\boldsymbol{K}$ from computational science [35], with dimension $N = 4$:

$$\underbrace{\begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}}_{\boldsymbol{K}} \underbrace{\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix}}_{\boldsymbol{u}} = \underbrace{\begin{bmatrix} 2 \\ 0 \\ 2 \\ 5 \end{bmatrix}}_{\boldsymbol{f}} \tag{37}$$

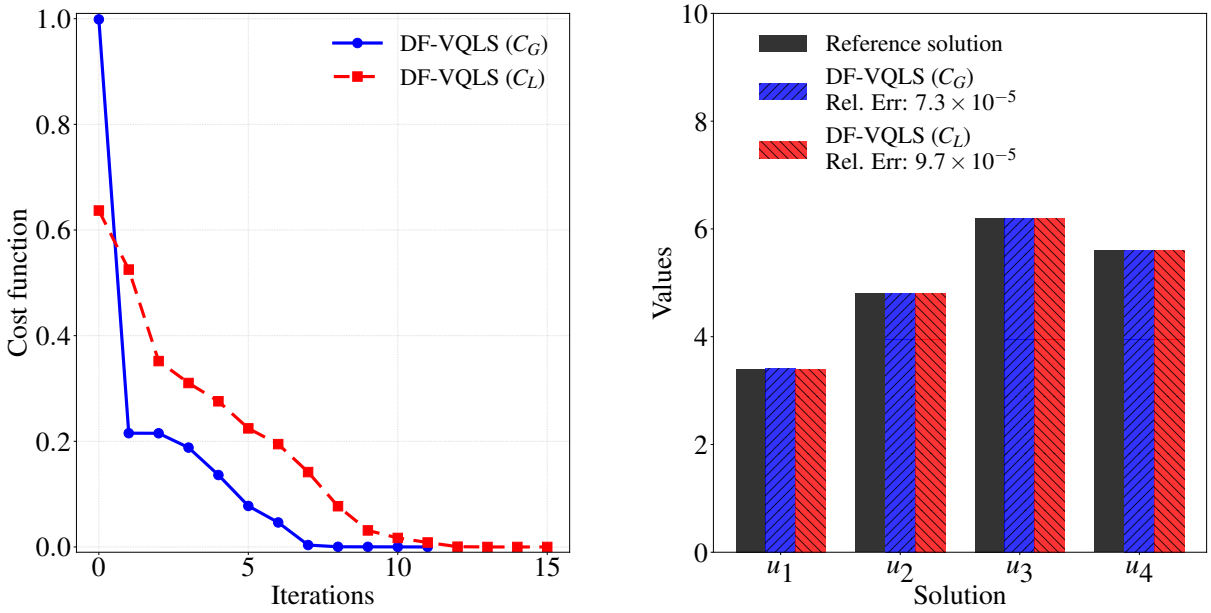


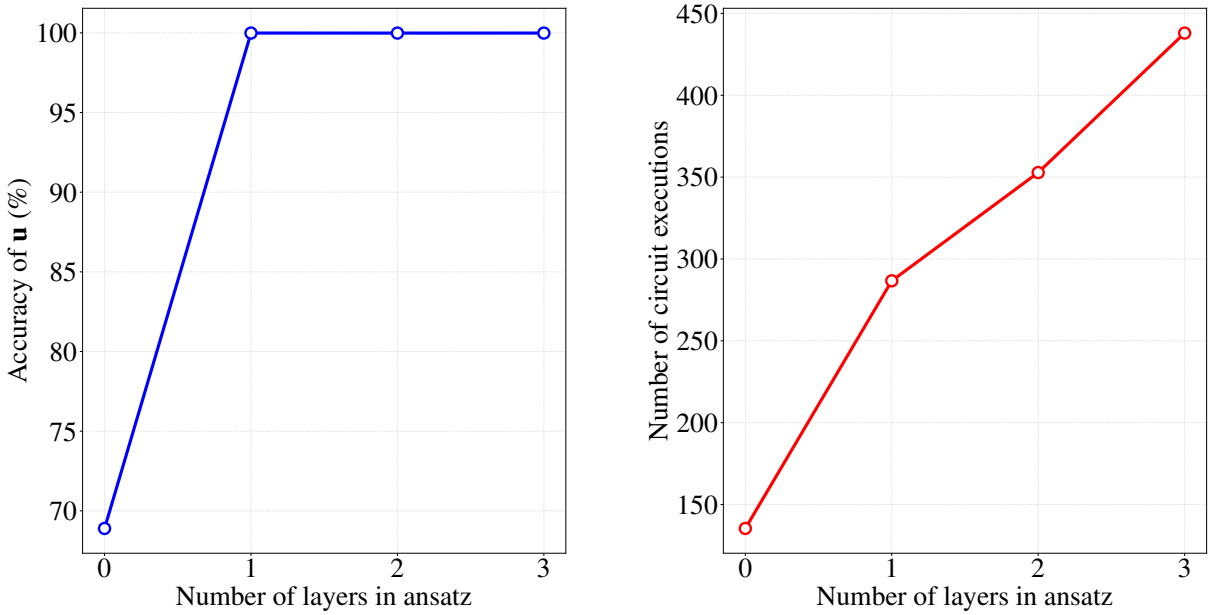
Figure 6: DF-VQLS results for solving Eq. (37), demonstrating both C_G and C_L cost functions. (a) Cost function evolutions during optimization. (b) Comparison between DF-VQLS solutions and the reference solution.

We evaluate solution accuracy using the relative l_2 norm error $\|\boldsymbol{u} - \boldsymbol{u}_{\text{ref}}\|/\|\boldsymbol{u}_{\text{ref}}\|$, where $\boldsymbol{u}_{\text{ref}}$ denotes the reference solution. For clarity, we may also express accuracy as $(1-\|\boldsymbol{u}-\boldsymbol{u}_{\text{ref}}\|/\|\boldsymbol{u}_{\text{ref}}\|)\times 100\%$. To solve this linear system, DF-VQLS requires 9 qubits for computing the numerators of C_G and C_L, and 13 qubits for their denominators. Crucially, it does not need to decompose the matrix $\boldsymbol{K}$ into an LCU.

Figure 6 presents the DF-VQLS results for solving Eq. (37), demonstrating both C_G and C_L cost functions. Using a 2-layer ansatz and BFGS convergence criterion $gtol = 10^{-3}$, Figure 6(a) shows C_G converging after 11 iterations and C_L after 15 iterations, with both final costs approaching zero. Figure 6(b) reveals excellent agreement between both solutions and the reference, achieving relative errors of 7.3×10^{-5} (C_G) and 9.7×10^{-5} (C_L). These results confirm the convergence and accuracy of DF-VQLS in solving the linear systems without matrix decomposition.

Figure 7 further examines the impact of ansatz layer count on DF-VQLS performance for solving Eq. (37). For the sake of simplicity, only the cost function C_L is presented here. We test layer counts from 0 to 3, where 0 layers corresponds to a single $\boldsymbol{R}_y$ gate layer (see Figure 2), producing only product states. For statistical robustness, each layer configuration includes 30 samples, and the averaged results are presented. Figure 7(a) demonstrates that the 0-layer ansatz achieves only 70% accuracy, indicating the true solution lies outside its representable state space. Accuracy approaches 100% for layers ≥ 1, confirming convergence with respect to layer count. However, Figure 7(b) shows that this improvement comes at the cost of increased circuit executions, as more layers require additional parameters $\boldsymbol{\theta}$ for gradient evaluation during optimization. For this example, one layer provides an optimal accuracy-efficiency trade-off. While currently no general theoretical guidelines exist for layer selection, we empirically set the layer count around $\log_2 N$ for the rest of this work.

The second example compares DF-VQLS with the original VQLS using a dense $N = 32$



(a) (b)

Figure 7: Ansatz layer convergence analysis for DF-VQLS in solving Eq. (37). (a) Solution accuracy versus ansatz layers. (b) Circuit executions versus ansatz layers.

system:

$$\underbrace{\begin{bmatrix} 5 & 0.5 & 0.5 & \cdots & 0.5 & 0.5 \\ 0.5 & 5 & 0.5 & \cdots & 0.5 & 0.5 \\ 0.5 & 0.5 & 5 & \cdots & 0.5 & 0.5 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0.5 & 0.5 & 0.5 & \cdots & 5 & 0.5 \\ 0.5 & 0.5 & 0.5 & \cdots & 0.5 & 5 \end{bmatrix}}_{\boldsymbol{K}\in\mathbb{R}^{32\times 32}} \underbrace{\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{31} \\ u_{32} \end{bmatrix}}_{\boldsymbol{u}} = \underbrace{\begin{bmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 1 \\ 1 \end{bmatrix}}_{\boldsymbol{f}} \tag{38}$$

Since the original VQLS requires decomposing $\boldsymbol{K}$ into an LCU, we implement the H2ZIXY decomposition method [15], previously used by Trahan et al. [8], yielding $k = 32$ unitary terms:

$$\begin{aligned} \boldsymbol{K} \;=\; & 5.0\,\boldsymbol{IIIII} + 0.5\,\boldsymbol{IIIIX} + 0.5\,\boldsymbol{IIIXI} + 0.5\,\boldsymbol{IIIXX} \\ & + 0.5\,\boldsymbol{IIXII} + 0.5\,\boldsymbol{IIXIX} + 0.5\,\boldsymbol{IIXXI} + 0.5\,\boldsymbol{IIXXX} \\ & + 0.5\,\boldsymbol{IXIII} + 0.5\,\boldsymbol{IXIIX} + 0.5\,\boldsymbol{IXIXI} + 0.5\,\boldsymbol{IXIXX} \\ & + 0.5\,\boldsymbol{IXXII} + 0.5\,\boldsymbol{IXXIX} + 0.5\,\boldsymbol{IXXXI} + 0.5\,\boldsymbol{IXXXX} \\ & + 0.5\,\boldsymbol{XIIII} + 0.5\,\boldsymbol{XIIIX} + 0.5\,\boldsymbol{XIIXI} + 0.5\,\boldsymbol{XIIXX} \\ & + 0.5\,\boldsymbol{XIXII} + 0.5\,\boldsymbol{XIXIX} + 0.5\,\boldsymbol{XIXXI} + 0.5\,\boldsymbol{XIXXX} \\ & + 0.5\,\boldsymbol{XXIII} + 0.5\,\boldsymbol{XXIIX} + 0.5\,\boldsymbol{XXIXI} + 0.5\,\boldsymbol{XXIXX} \\ & \underbrace{+0.5\,\boldsymbol{XXXII} + 0.5\,\boldsymbol{XXXIX} + 0.5\,\boldsymbol{XXXXI} + 0.5\,\boldsymbol{XXXXX}}_{\text{32 terms in this LCU}} \end{aligned} \tag{39}$$

For the sake of simplicity, we only use the local cost function C_L in both the original VQLS and the proposed DF-VQLS in the following numerical examples. The original VQLS requires 6

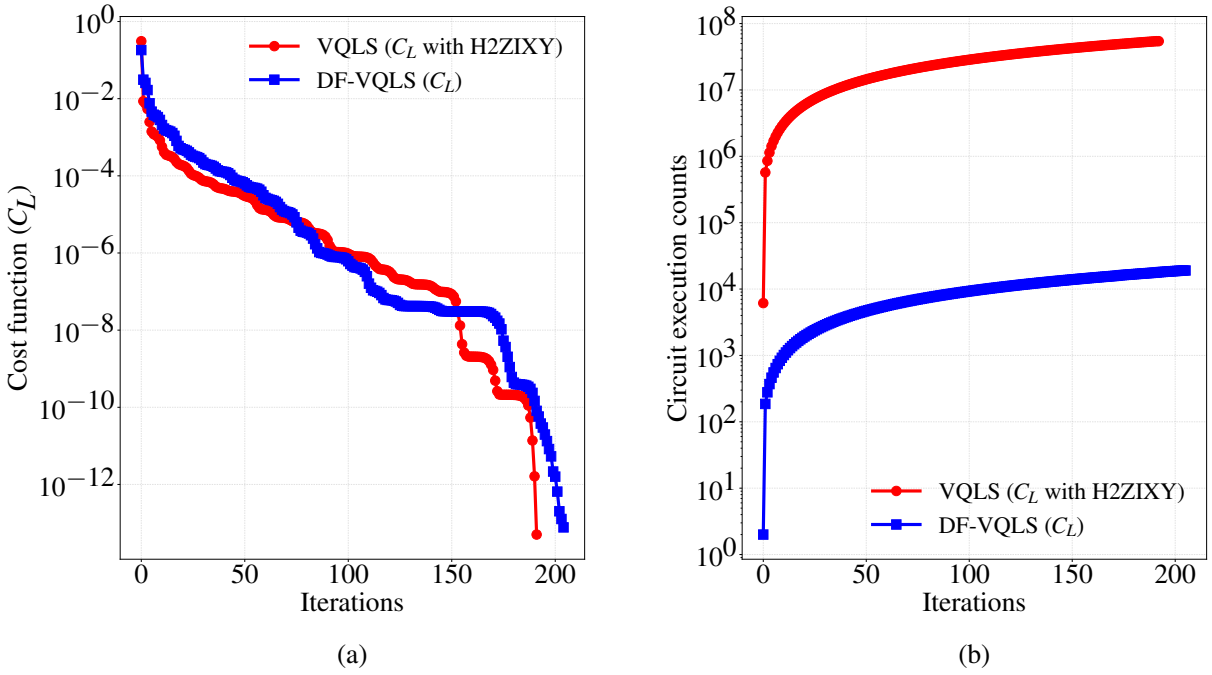


Figure 8: Performance comparison between DF-VQLS and original VQLS for solving Eq. (38), where the local cost function C_L is adopted. (a) Cost function evolution curves. (b) Circuit execution counts.

qubits for both numerator and denominator computations in C_L. Note that a critical limitation of the original VQLS arises from the decomposition in Eq. (39): it necessitates $k^2(n_q+1) = 6144$ circuit executions per cost function evaluation [6]. As a comparison, DF-VQLS requires 21 and 31 qubits for the numerator and denominator, respectively. This qubit count is larger but maintains $\mathcal{O}(\log N)$ scaling. Crucially, DF-VQLS needs only 2 circuit executions per cost evaluation, representing a substantial reduction from the 6144 of the original VQLS.

Figure 8 compares DF-VQLS and original VQLS performance on solving Eq. (38), using identical 5-layer ansatz and $gtol = 10^{-8}$ convergence criteria. Figure 8(a) shows both methods achieve similar cost trajectories, reaching below 10^{-12} after around 200 iterations. Solution accuracy is comparable, with relative errors of 1.84×10^{-6} (DF-VQLS) and 1.43×10^{-6} (VQLS). This equivalence stems from DF-VQLS preserving the C_L definition while modifying only its quantum circuit implementation via vectorization and swap test. However, Figure 8(b) reveals the clear advantage of DF-VQLS in circuit efficiency: 19,046 executions versus 54,269,952 from the original VQLS, which is a nearly 99.9965% reduction. This improvement arises because DF-VQLS avoids the quadratic scaling of the original VQLS with decomposition terms, requiring just 2 executions per cost evaluation.

Note that all simulations in this work were performed on a noiseless simulator. This allows for the exact calculation of gradients, making gradient-based optimizers like BFGS an efficient choice. In our tests, we observed that a smaller convergence tolerance *gtol* generally leads to higher solution accuracy at the cost of more optimization iterations. We also confirmed that gradient-free optimizers, such as SPSA [18, 36] and COBYLA, can also be used in DF-VQLS, while they may require more iterations than BFGS. However, on real quantum hardware, the performance would be fundamentally different. The presence of gate errors and measurement shot noise reduces the accuracy of the estimation of the cost function and its gradients. As shown by Wang et al. [37], such noise can cause the optimization landscape to flatten, a phenomenon known as a noise-induced barren plateau. In this context, the robustness of the classical opti-

mizer becomes important. Gradient-free methods like SPSA are often more resilient to noise than gradient-based methods that rely on precise gradient information [18]. A more detailed comparison of various optimizers can be found in [38].

In summary, numerical examples on the quantum simulator demonstrate the ability of the proposed DF-VQLS to accurately solve linear systems without matrix decomposition. The comparison with the original VQLS highlights the notable reduction of DF-VQLS in circuit executions while maintaining equivalent convergence and accuracy.

4 Application examples in computational mechanics

In this section, we present three application examples of DF-VQLS in computational mechanics, including bar, truss, and two-dimensional continuum problems. In addition, a discussion on the usage of the preconditioning method to improve the performance of DF-VQLS is shown in Section 4.2. For the sake of simplicity, only the DF-VQLS with the local cost function C_L is used. All the numerical examples are carried out on the quantum simulator.

4.1 Bar and truss

In this subsection, we present two application examples of the proposed DF-VQLS, including bar and truss problems.

The first example is a linear elastic bar under distributed loading, adapted from Dolbow and Belytschko [39]. As shown in Figure 9, the bar has length $L = 1$ mm, Young's modulus $E = 10$ MPa, and uniform cross-sectional area $A = 1$ mm^2. The bar is fixed at $x = 0$ and subjected to a linearly varying body force $f(x) = x$ N/mm. The governing equations are:

$$EA\frac{d^2u}{dx^2} + f(x) = 0, \quad 0 < x < L \tag{40a}$$

$$u(x = 0) = 0 \tag{40b}$$

$$\frac{du}{dx}(x = L) = 0 \tag{40c}$$

where $u(x)$ represents the axial displacement field. The analytical solution is:

$$u_{\text{ref}}(x) = \frac{1}{EA}\left(\frac{L^2x}{2} - \frac{x^3}{6}\right) = \frac{x}{20} - \frac{x^3}{60} \quad \text{(mm)} \tag{41}$$

To solve Eq. (40) using DF-VQLS, we discretize the domain using the finite element method with linear shape functions and 2-node bar elements. After applying the boundary conditions, we obtain the linear system $\boldsymbol{K}\boldsymbol{u} = \boldsymbol{f}$, where $\boldsymbol{K}$ is the global stiffness matrix, $\boldsymbol{u}$ is the nodal displacement vector, and $\boldsymbol{f}$ is the nodal force vector. This system is solved using the proposed DF-VQLS without matrix decomposition.

Figure 10 presents DF-VQLS solutions for the bar problem with three discretizations ($N = 4$, 8, and 16 elements). Using 4 ansatz layers and a convergence tolerance of $gtol = 10^{-20}$, all cases show close agreement with the analytical reference solution in Figure 10(a). The relative errors in Figure 10(b), computed as $|u(x) - u_{\text{ref}}(x)|/|u_{\text{ref}}(x)|$, demonstrate 10^{-7} magnitude accuracy for $N = 4$ and 8. While the $N = 16$ case exhibits increased error, it remains below 10^{-5}. These results validate the capability of DF-VQLS for solving the bar problem.

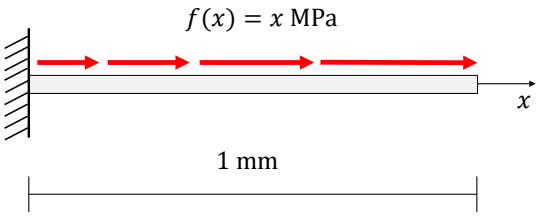


Figure 9: Linear elastic bar under distributed loading: geometry and boundary conditions.

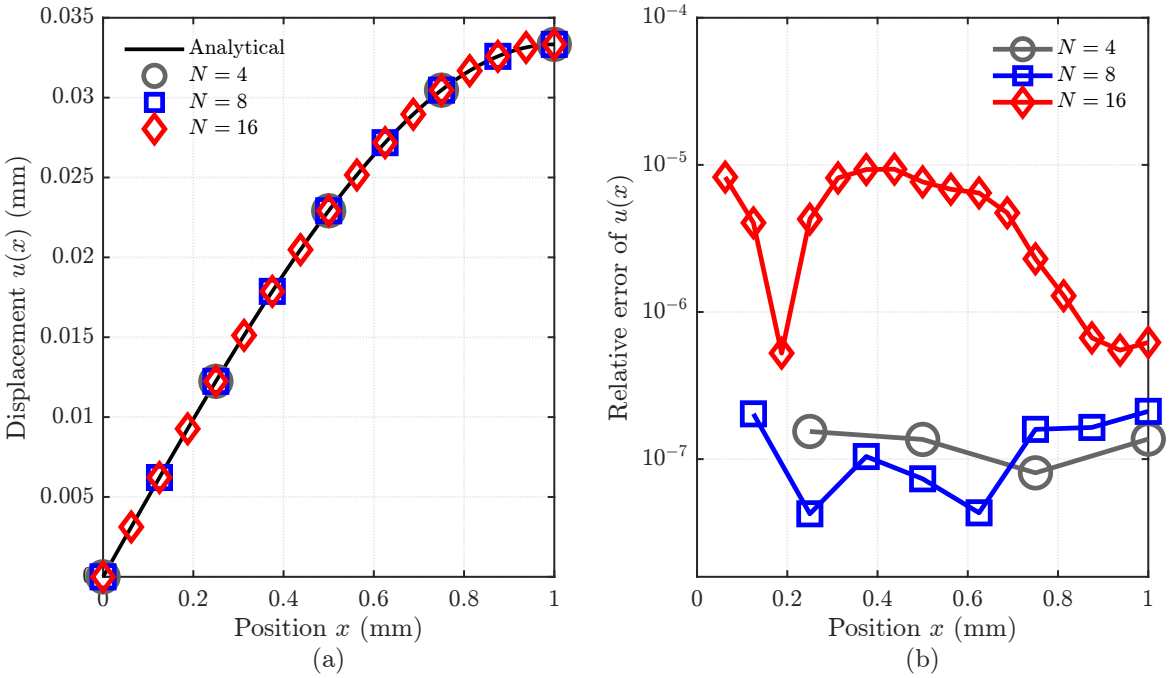


Figure 10: Results of the bar problem solved by the DF-VQLS, where three problem sizes $N = 4, 8,$ and 16 are presented. (a) Displacement $u(x)$. (b) Relative error of $u(x)$, compared with the analytical solution.

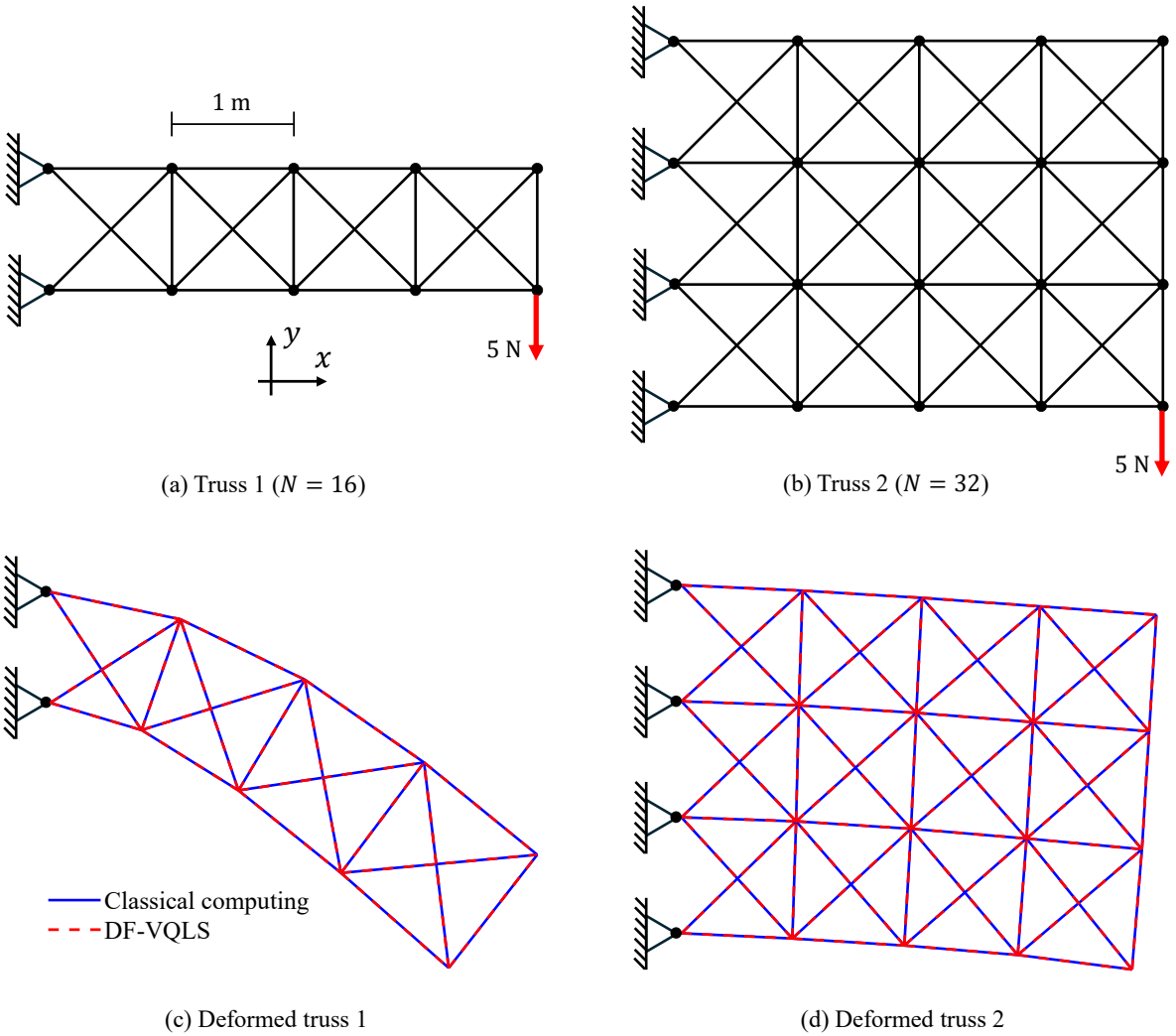
The second example involves two truss structures with the geometry shown in Figure 11(a-b). All truss members have a cross-sectional area $A = 10^{-4}$ m^2 and Young's modulus $E = 10$ MPa. The left-side nodes are fully constrained ($u_x = u_y = 0$), and a concentrated force $\boldsymbol{P} = (0, -5)$ N is applied at the lower-right corner nodes. The global stiffness matrix is assembled through direct summation of element contributions:

$$\boldsymbol{K} = \sum_{e=1}^{m} \frac{EA}{L_e} \boldsymbol{h}_e \boldsymbol{h}_e^T \tag{42}$$

where for each element e:

- $L_e = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$ is the element length
- $\boldsymbol{h}_e = [-\cos\theta_e, -\sin\theta_e, \cos\theta_e, \sin\theta_e]^T$ is the direction vector
- $\theta_e = \arctan\left(\frac{y_j - y_i}{x_j - x_i}\right)$ is the element orientation angle

After constraining the left-side nodes and applying the external force, the dimensions of the



(a) Truss 1 ($N = 16$)

(b) Truss 2 ($N = 32$)

(c) Deformed truss 1

(d) Deformed truss 2

Figure 11: Two truss structures simulated using DF-VQLS, with classical computing results shown as reference. (a) Truss 1 with problem size $N = 16$: geometry and boundary conditions. (b) Truss 2 with problem size $N = 32$: geometry and boundary conditions. (c) Deformed truss 1 (10 times scaled). (d) Deformed truss 2 (10 times scaled).

linear systems $\boldsymbol{K}\boldsymbol{u} = \boldsymbol{f}$ are $N = 16$ and 32 for the first and second truss structures, respectively. These linear systems are then solved using DF-VQLS.

Figure 11(c-d) present DF-VQLS solutions for the truss problems, with classical computing results shown as reference. Both problems use 8 ansatz layers and a convergence tolerance of $gtol = 10^{-20}$. For clear visualization, the obtained displacement $\boldsymbol{u}$ is scaled by a factor of 10. The DF-VQLS results show excellent agreement with the reference solutions, both achieving an overall relative error less than 10^{-4}.

While DF-VQLS achieves moderate accuracy in these examples, classical linear solvers (e.g., Gaussian elimination, conjugate gradient) typically attain machine precision ($< 10^{-16}$) due to deterministic algorithms and high-precision arithmetic. The accuracy limitation of DF-VQLS arises from many factors, including the finite expressibility of the ansatz and, more critically, challenges related to the barren plateau phenomenon, making gradient-based optimization difficult [32]. This issue is related to the expressibility of the ansatz. Highly expressive circuits,

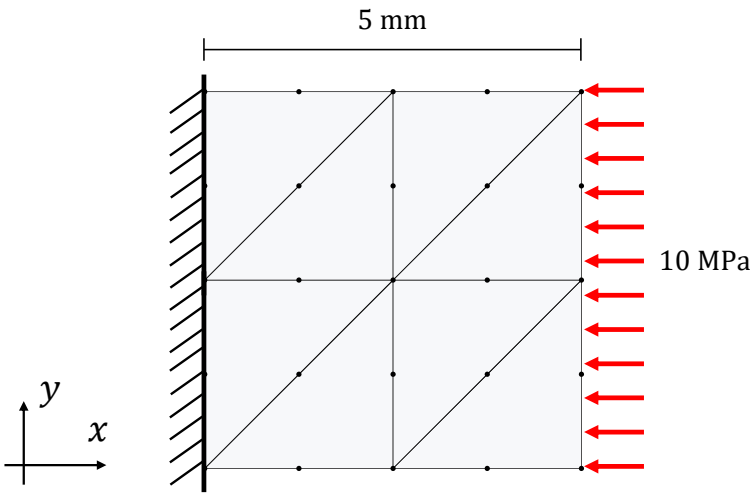


Figure 12: Schematic of the plate problem: A fixed left edge, uniformly pressurized right edge, and mesh discretization using eight 6-node triangular elements.

related to large problem size and deep ansatz layers, are more prone to barren plateaus because they explore a vast parameter space in an unstructured manner [40]. This creates a fundamental trade-off: an ansatz must be expressive enough to contain the solution, but making it harder to train. Recent work on the iterative refinement method [41] provides an approach to enhance the final solution accuracy, warranting further investigation.

4.2 Two-dimensional continuum problem

In this subsection, we demonstrate the application of DF-VQLS to a two-dimensional continuum solid mechanics problem, where the role of preconditioning is also discussed.

As illustrated in Figure 12, the problem involves a square plate fixed on the left edge and subjected to uniform pressure on the right edge. The material is linear elastic under plane stress conditions, with Young's modulus $E = 3 \times 10^5$ MPa and Poisson's ratio $\nu = 0.3$. Neglecting body force and using finite element formulations, the governing equations are derived from the principle of virtual work:

$$\int_{\Omega} \boldsymbol{\sigma} : \delta\boldsymbol{\epsilon}, d\Omega = \int_{\Gamma_N} \boldsymbol{t} \cdot \delta\boldsymbol{u}, d\Gamma, \tag{43}$$

where $\boldsymbol{\sigma} = \boldsymbol{C}\boldsymbol{\epsilon}$ is the Cauchy stress, $\boldsymbol{\epsilon} = \nabla^s \boldsymbol{u}$ is the strain, $\delta\boldsymbol{u}$ is the virtual displacement field, and $\boldsymbol{t}$ is the prescribed traction on the Neumann boundary Γ_N. The plate is discretized using eight 6-node triangular elements. After applying boundary conditions, the linear system $\boldsymbol{K}\boldsymbol{u} = \boldsymbol{f}$ is obtained with dimension $N = 50$. Since quantum state encoding requires N to be a power of 2, we pad $\boldsymbol{K}$ and $\boldsymbol{f}$ to $N = 64$. Specifically, $\boldsymbol{K}$ is extended with zeros and its new diagonal entries are set to $\max(|\boldsymbol{K}|)$, while $\boldsymbol{f}$ is padded with zeros. This linear system is then solved using the proposed DF-VQLS.

The stiffness matrix $\boldsymbol{K}$ has a condition number $\kappa(\boldsymbol{K}) = 2.95 \times 10^6$, which is not hard to handle for classical linear solvers. However, as shown in [6], the VQLS cost function C_L must

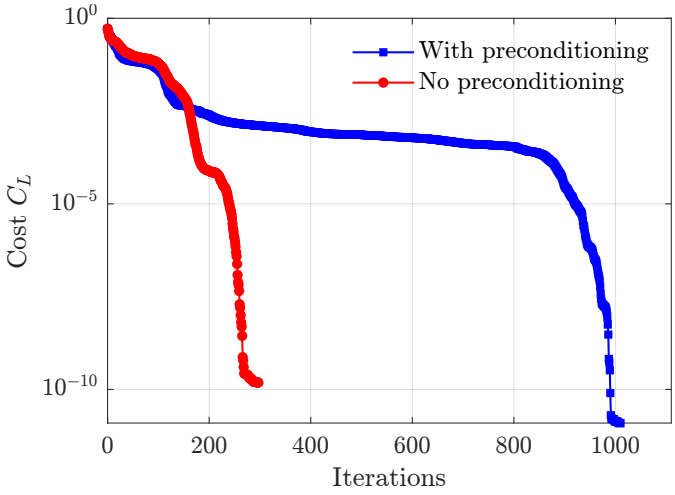


Figure 13: Evolution of the cost function C_L for DF-VQLS with and without Jacobi preconditioning.

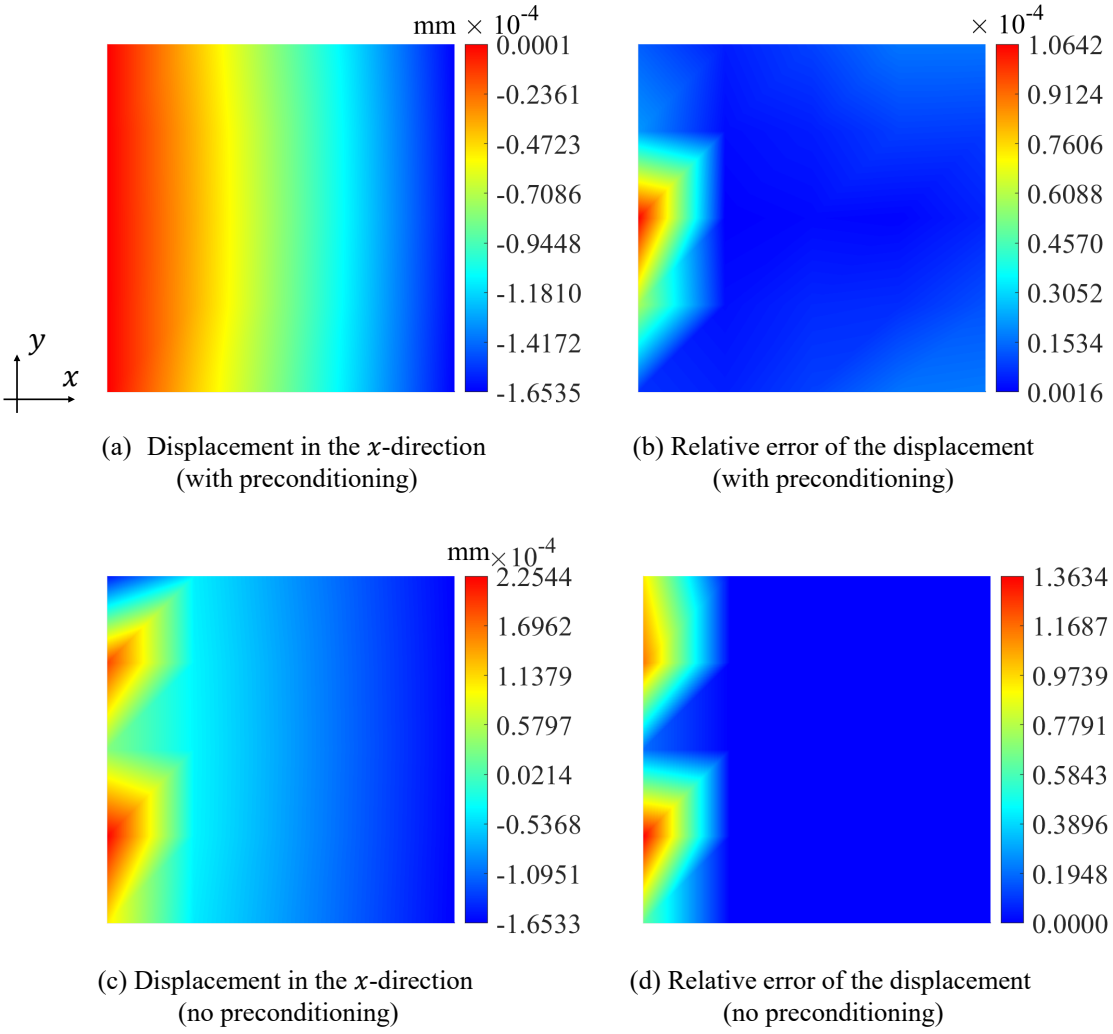


Figure 14: Displacement fields (x-direction) for (a) preconditioned and (c) unpreconditioned solutions. Corresponding relative errors are shown in (b) and (d).

satisfy:

$$C_L \geq \frac{\epsilon_{\text{acc}}^2}{\kappa(\boldsymbol{K})^2 n_q}, \tag{44}$$

where ϵ_{acc} is the target solution precision and $n_q = 6$. For an assumed precision $\epsilon_{\text{acc}} = 0.01$, achieving the corresponding C_L around 1.91×10^{-18} seems infeasible even on a noiseless quantum simulator. To address this, we implement the Jacobi preconditioning method [42], with the preconditioned system defined as:

$$\boldsymbol{K}_{\text{pre}} = \boldsymbol{D}^{-1/2} \boldsymbol{K} \boldsymbol{D}^{-1/2}, \quad \boldsymbol{f}_{\text{pre}} = \boldsymbol{D}^{-1/2} \boldsymbol{f}, \tag{45}$$

where $\boldsymbol{D}$ is the diagonal matrix of $\boldsymbol{K}$. Solving $\boldsymbol{K}_{\text{pre}} \boldsymbol{u}_{\text{pre}} = \boldsymbol{f}_{\text{pre}}$ and recovering $\boldsymbol{u} = \boldsymbol{D}^{-1/2} \boldsymbol{u}_{\text{pre}}$ reduces the condition number $\kappa(\boldsymbol{K}_{\text{pre}})$ to 4.29×10^2, an improvement factor of 6.88×10^3.

Figure 13 shows the convergence of C_L for both preconditioned and unpreconditioned systems. Both approaches use 10 ansatz layers and a convergence tolerance of $gtol = 10^{-20}$. Despite both reaching $C_L \approx 10^{-10}$, the unpreconditioned solution yields an overall relative error of 0.74, while preconditioning achieves 5.30×10^{-5}. This stark contrast is evident in the displacement fields shown in Figure 14, where the preconditioned solution (see Figure 14(a-b)) shows a maximum relative error of 1.06×10^{-4}. In comparison, the unpreconditioned result (see Figure 14(c-d)) exhibits errors exceeding 1.36. The reason for this significant improvement is that preconditioning effectively reduces the system's condition number κ, which, as indicated by Eq. (44), directly impacts the required precision of the cost function for a given solution accuracy.

Regarding the classical cost of the Jacobi preconditioning method, constructing and applying this preconditioner takes $\mathcal{O}(N)$ for a sparse system. From the perspective of a pure complexity analysis, the overall complexity of DF-VQLS may become $\mathcal{O}(N) + \mathcal{O}(\text{polylog}(N)) = \mathcal{O}(N)$, i.e., the complexity is dominated by the preconditioning. However, the cost of preconditioning is actually polynomially smaller than the complexity of the classical solver, i.e., the conjugate gradient method with complexity $\mathcal{O}(N\sqrt{\kappa})$, where in typical finite element problems $\kappa \sim \mathcal{O}(\text{poly}(N))$ [2]. Therefore, this modest classical pre-processing step may not compromise the overall potential speedup offered by DF-VQLS. Additionally, we also notice that investigating preconditioners is an active area of research for quantum linear solvers [43–47]. Exploring more sophisticated, quantum-compatible preconditioners is a promising direction for future research. For instance, the block-diagonal, multigrid-inspired approaches [48], or the quantum circulant preconditioner [44], could also be integrated with DF-VQLS in future work for addressing ill-conditioned systems.

5 Conclusion

This work proposed a decomposition-free variational quantum linear solver (DF-VQLS). It aims to overcome a critical limitation of the original VQLS: in general cases, the coefficient matrix in a linear system cannot be efficiently decomposed into a linear combination of unitaries. The key innovation of DF-VQLS lies in the proposal of two vectorization techniques, which map the cost functions of the original VQLS to the inner product of vectors. In this way, the coefficient matrix only needs to be reformulated as a vector, effectively eliminating the need for matrix decomposition. Meanwhile, another key advantage is that only two quantum circuits are required to be executed per cost function evaluation, which represents a notable

reduction compared to the original VQLS. Numerical examples, including three applications in computational mechanics problems, were conducted on the quantum simulator, verifying the convergence and accuracy of the proposed method.

Future work will explore a proof-of-concept implementation of DF-VQLS on a real quantum computer to validate its noise resilience and scalability. Furthermore, the proposed vectorization techniques can be directly adapted to the variational quantum eigensolver (VQE) [49, 50], bypassing the need for Hamiltonian decomposition in eigenvalue problems. This suggests potential decomposition-free quantum algorithms in fields such as computational chemistry and materials science, where high-dimensional eigenvalue equations are prevalent. It is expected that the spirit of decomposition-free quantum computing could be extended to broader fields.

It is also important to clarify that the application of quantum computing to computational mechanics is believed to be at its early stage [51]. While quantum computing has the ability to surpass classical computing in certain computational tasks in theory, a solid and provable demonstration for practically useful tasks is still open. To achieve a quantum advantage for realistic computational mechanics problems, many challenging problems are still to be explored, e.g., better hardware, efficient state preparation, readout problem [26], end-to-end complexity advantage [52], and many more. Although the practical value is not achieved yet, a valid point is that quantum computing now indeed provides a new paradigm to advance computational mechanics and a different aspect to rethink about computation itself.

Acknowledgements

This work has been supported by the National Key R&D Program of China (Grant No. 2022YFE0113100) and the National Natural Science Foundation of China (Grant Nos. 12432009, 11920101002). The authors sincerely thank Boyang Hu at Qiuzhen College, Tsinghua University, for his assistance in proving the vectorization theorems.

References

[1] G. Strang, Linear algebra and its applications, 2000.

[2] J. R. Shewchuk, et al., An introduction to the conjugate gradient method without the agonizing pain (1994).

[3] M. E. Morales, L. Pira, P. Schleich, K. Koor, P. Costa, D. An, L. Lin, P. Rebentrost, D. W. Berry, Quantum linear system solvers: A survey of algorithms and applications, arXiv preprint arXiv:2411.02522 (2024).

[4] A. W. Harrow, A. Hassidim, S. Lloyd, Quantum algorithm for linear systems of equations, Physical Review Letters 103 (15) (2009) 150502.

[5] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, et al., Variational quantum algorithms, Nature Reviews Physics 3 (9) (2021) 625–644.

[6] C. Bravo-Prieto, R. LaRose, M. Cerezo, Y. Subasi, L. Cincio, P. J. Coles, Variational quantum linear solver, Quantum 7 (2023) 1188.

[7] M. Ali, M. Kabel, Performance study of variational quantum algorithms for solving the poisson equation on a quantum computer, Physical Review Applied 20 (1) (2023) 014054.

[8] C. J. Trahan, M. Loveland, N. Davis, E. Ellison, A variational quantum linear solver application to discrete finite-element methods, Entropy 25 (4) (2023) 580.

[9] G. T. Balducci, B. Chen, M. Möller, R. De Breuker, Solving 1D Poisson problem with a variational quantum linear solver, arXiv preprint arXiv:2412.04938 (2024).

[10] A. Arora, B. M. Ward, C. Oskay, An implementation of the finite element method in hybrid classical/quantum computers, Finite Elements in Analysis and Design 248 (2025) 104354.

[11] A. Gnanasekaran, A. Surana, Efficient variational quantum linear solver for structured sparse matrices, in: 2024 IEEE International Conference on Quantum Computing and Engineering (QCE), Vol. 1, IEEE, 2024, pp. 199–210.

[12] M. A. Nielsen, I. L. Chuang, Quantum computation and quantum information, 10th Edition, Cambridge University Press, USA, 2011.

[13] Y. Sato, R. Kondo, S. Koide, H. Takamatsu, N. Imoto, Variational quantum algorithm based on the minimum potential energy for solving the poisson equation, Physical Review A 104 (5) (2021) 052409.

[14] H.-L. Liu, Y.-S. Wu, L.-C. Wan, S.-J. Pan, S.-J. Qin, F. Gao, Q.-Y. Wen, Variational quantum algorithm for the poisson equation, Physical Review A 104 (2) (2021) 022418.

[15] R. M. N. Pesce, P. D. Stevenson, H2ZIXY: Pauli spin matrix decomposition of real symmetric matrices, arXiv preprint arXiv:2111.00627 (2021).

[16] H. Buhrman, R. Cleve, J. Watrous, R. De Wolf, Quantum fingerprinting, Physical Review Letters 87 (16) (2001) 167902.

[17] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, J. M. Gambetta, Quantum computing with Qiskit, arXiv preprint arXiv:2405.08810 (2024).

[18] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow, J. M. Gambetta, Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets, Nature 549 (7671) (2017) 242–246.

[19] S. J. Wright, Numerical optimization (2006).

[20] M. J. Powell, Direct search algorithms for optimization calculations, Acta Numerica 7 (1998) 287–336.

[21] H. V. Henderson, S. R. Searle, Vec and vech operators for matrices, with some uses in jacobians and multivariate statistics, Canadian Journal of Statistics 7 (1) (1979) 65–81.

[22] W. Roth, On direct product matrices, Bull. Amer. Math. Soc. 40 (12) (1934) 461–468.

[23] Y. Xu, J. Yang, Z. Kuang, Q. Huang, W. Huang, H. Hu, Quantum computing enhanced distance-minimizing data-driven computational mechanics, Computer Methods in Applied Mechanics and Engineering 419 (2024) 116675.

[24] A. S. Householder, Unitary triangularization of a nonsymmetric matrix, Journal of the ACM (JACM) 5 (4) (1958) 339–342.

[25] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, S. Lloyd, Quantum machine learning, Nature 549 (7671) (2017) 195–202.

[26] S. Aaronson, Read the fine print, Nature Physics 11 (4) (2015) 291–293.

[27] V. Giovannetti, S. Lloyd, L. Maccone, Quantum random access memory, Physical Review Letters 100 (16) (2008) 160501.

[28] X.-M. Zhang, T. Li, X. Yuan, Quantum state preparation with optimal circuit depth: Implementations and applications, Physical Review Letters 129 (23) (2022) 230504.

[29] J. Preskill, Quantum computing in the NISQ era and beyond, Quantum 2 (2018) 79.

[30] P. Murali, D. C. McKay, M. Martonosi, A. Javadi-Abhari, Software mitigation of crosstalk on noisy intermediate-scale quantum computers, in: Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems, 2020, pp. 1001–1016.

[31] H.-Y. Huang, R. Kueng, J. Preskill, Predicting many properties of a quantum system from very few measurements, Nature Physics 16 (10) (2020) 1050–1057.

[32] M. Larocca, S. Thanasilp, S. Wang, K. Sharma, J. Biamonte, P. J. Coles, L. Cincio, J. R. McClean, Z. Holmes, M. Cerezo, Barren plateaus in variational quantum computing, Nature Reviews Physics (2025) 1–16.

[33] R. Iten, R. Colbeck, I. Kukuljan, J. Home, M. Christandl, Quantum circuits for isometries, Physical Review A 93 (3) (2016) 032318.

[34] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, SciPy 1.0 Contributors, SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python, Nature Methods 17 (2020) 261–272.

[35] G. Strang, Computational science and engineering, SIAM, 2007.

[36] J. C. Spall, Multivariate stochastic approximation using a simultaneous perturbation gradient approximation, IEEE transactions on automatic control 37 (3) (2002) 332–341.

[37] S. Wang, E. Fontana, M. Cerezo, K. Sharma, A. Sone, L. Cincio, P. J. Coles, Noise-induced barren plateaus in variational quantum algorithms, Nature Communications 12 (1) (2021) 6961.

[38] A. Pellow-Jarman, I. Sinayskiy, A. Pillay, F. Petruccione, A comparison of various classical optimizers for a variational quantum linear solver, Quantum Information Processing 20 (6) (2021) 202.

[39] J. Dolbow, T. Belytschko, An introduction to programming the meshless element free Galerkin method, Archives of computational methods in engineering 5 (1998) 207–241.

[40] Z. Holmes, K. Sharma, M. Cerezo, P. J. Coles, Connecting ansatz expressibility to gradient magnitudes and barren plateaus, PRX quantum 3 (1) (2022) 010313.

[41] Y. Saito, X. Lee, D. Cai, J. Shin, N. Asai, Iterative refinement for variational quantum linear solver, in: International Conference on Data Analytics and Insights, Springer, 2023, pp. 15–27.

[42] E. Turkel, Preconditioning techniques in computational fluid dynamics, Annual Review of Fluid Mechanics 31 (1) (1999) 385–416.

[43] B. D. Clader, B. C. Jacobs, C. R. Sprouse, Preconditioned quantum linear system algorithm, Physical Review Letters 110 (25) (2013) 250504.

[44] C. Shao, H. Xiang, Quantum circulant preconditioner for a linear system of equations, Physical Review A 98 (6) (2018) 062321.

[45] Y. Tong, D. An, N. Wiebe, L. Lin, Fast inversion, preconditioned quantum linear system solvers, fast green's-function computation, and fast evaluation of matrix functions, Physical Review A 104 (3) (2021) 032422.

[46] J. Golden, D. O'Malley, H. Viswanathan, Quantum computing and preconditioners for hydrological linear systems, Scientific Reports 12 (1) (2022) 22285.

[47] S. Jin, N. Liu, C. Ma, Y. Yu, Quantum preconditioning method for linear systems problems via Schrödingerization, arXiv preprint arXiv:2505.06866 (2025).

[48] G. H. Golub, C. F. Van Loan, Matrix computations, JHU press, 2013.

[49] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, J. L. O'brien, A variational eigenvalue solver on a photonic quantum processor, Nature Communications 5 (1) (2014) 4213.

[50] J. Tilly, H. Chen, S. Cao, D. Picozzi, K. Setia, Y. Li, E. Grant, L. Wossnig, I. Rungger, G. H. Booth, et al., The variational quantum eigensolver: a review of methods and best practices, Physics Reports 986 (2022) 1–128.

[51] Y. Xu, Z. Kuang, Q. Huang, J. Yang, H. Hu, Quantum computing: The new focus in computational mechanics, Advances in Mechanics 55 (2) (2025) 419–430.

[52] L. Lin, Quantum advantages and end-to-end complexity, Collections 57 (03) (2024).